



S⁴D: Style and Structure Similarity for Site Domains

S⁴D: Style and Structure Similarity for Site Domains: A system for website structure and visual style analysis, clustering, and comparison

Panagiotis Favvatas

Master's Thesis, Department of Informatics And Telecommunications
University of Peloponnese, June 2025

Copyright © 2025 University of Peloponnese. All Rights Reserved

**S⁴D: Style and Structure Similarity for Site Domains:
A
system for website structure and visual style analysis,
clustering, and comparison**

Panagiotis Favvatas

A thesis submitted in partial fulfillment
of the requirements for the MSc in
Computer Science

Committee

Professor, Spiros Skiadopoulos, Supervisor
Professor, [Committee Member 1]
Associate Professor, [Committee Member 2]

University of the Peloponnese
2025

Περίληψη

Αυτή η διπλωματική εργασία παρουσιάζει το **S⁴D** (Style and Structure Similarity for Site Domains), μια ερευνητική πλατφόρμα για την ανάλυση ομοιοτήτων μεταξύ ιστοσελίδων βάσει της δομής και του οπτικού τους στυλ. Το σύστημα επεξεργάζεται ιστοσελίδες αναλύοντας τη δομή του DOM και τις ιδιότητες CSS, συνδυάζοντάς τα σε μία ενιαία μορφή για συγκριτική αξιολόγηση.

Οι χρήστες μπορούν να εισάγουν οποιονδήποτε αριθμό URLs για ανάλυση. Κάθε σελίδα επεξεργάζεται ανεξάρτητα για την εξαγωγή δομικών και στυλιστικών χαρακτηριστικών. Κατά τη φάση υπολογισμού της ομοιότητας, οι συγκρίσεις πραγματοποιούνται σε επίπεδο domain, ανεξάρτητα από τον τρόπο ομαδοποίησης των URLs. Το σύστημα υποστηρίζει την ανάλυση σε διαφορετικά βάθη DOM (επίπεδα 1 έως 10).

Η ομαδοποίηση των στοιχείων πραγματοποιείται με τον αλγόριθμο HDBSCAN, ενώ η ποιότητα των ομάδων αξιολογείται με το DBCV. Τα απομονωμένα στοιχεία επανατοποθετούνται σε κατάλληλες ομάδες όταν είναι δυνατόν. Η πλατφόρμα επιτρέπει τη διεξαγωγή πολλαπλών πειραμάτων με διαφορετικά σύνολα URLs και παραμέτρους, καλύπτοντας ποι- κίλες ερευνητικές ανάγκες.

Το **S⁴D** υλοποιείται με React για το περιβάλλον χρήστη, Flask για τις διεπαφές API και Python για την επεξεργασία δεδομένων. Ενσωματώνει το Gnuplot για παραγωγή γραφημάτων και αρχείων κατάλληλων για ακαδημαϊκή χρήση. Ένα αρχείο αποτελεσμάτων επιτρέπει τη σύγκριση διαφορετικών πειραμάτων και την παρακολούθηση της προόδου.

Το σύστημα εξάγει HTML σελίδες με χρωματική κωδικοποίηση που αναδεικνύουν τις δομικές ομοιότητες, καθώς και πίνακες συσχετίσεων βασισμένους σε υπολογισμούς συνημιτόνου. Οι χρονικές μετρήσεις συμβάλλουν στην αξιολόγηση της απόδοσης του συστήματος. Οι δοκιμές δείχνουν ότι το **S⁴D** συγκρίνει και ομαδοποιεί με ακρίβεια στοιχεία ιστοσελίδων σε διαφορετικά domains, προσφέροντας ένα αξιόπιστο εργαλείο για δομική και οπτική ανάλυση.

Η πλήρης υλοποίηση του συστήματος είναι διαθέσιμη στο: https://github.com/pfavvatas/lib_url_to_img

Abstract

This thesis presents S⁴D (Style and Structure Similarity for Site Domains), a research platform for analyzing similarities between websites based on their structure and visual style. The system examines web pages by processing their DOM structure and CSS styling, combining both into a unified representation for comparison.

Users can provide any number of URLs for analysis. Each page is processed independently to extract structural and styling information. During the similarity analysis phase, comparisons are performed at the domain level—regardless of how the URLs were grouped initially. The system supports analysis at multiple DOM depths, from level 1 to level 10.

After grouping similar elements using HDBSCAN and evaluating cluster quality with DBCV, the system reassigns outliers to the most appropriate clusters when possible. Users can perform multiple experiments using different URL sets and DOM levels to support varied research scenarios.

S⁴D is built with React for the interface, Flask for backend APIs, and Python for processing logic. It integrates Gnuplot to generate charts and data suitable for academic analysis. A results archive enables users to compare and track experiment outputs over time.

The system outputs color-coded HTML pages that highlight structural similarities, as well as similarity matrices based on cosine similarity. Timing metrics help evaluate system performance. Testing confirms that S⁴D reliably groups and compares website content across domains, offering a robust tool for structural and visual web analysis.

The full system implementation is available at: https://github.com/pfavvatas/lib_url_to_img

Acknowledgements

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this thesis.

First, I would like to thank Professor Spiros Skiadopoulos for his invaluable scientific guidance and mentorship throughout the course of this thesis. His expertise, constructive feedback, and immediate assistance whenever needed have been instrumental in shaping this research. His dedication to academic excellence has significantly contributed to my development as a researcher.

Furthermore, I am grateful to the academic staff and fellow researchers who have enriched my learning experience through valuable discussions and collaborative exchange of ideas.

I am deeply thankful to my family for their unconditional support, encouragement, and understanding throughout this academic journey. Their constant belief in my abilities has provided me with the strength to overcome challenges and pursue my goals.

Finally, I would like to acknowledge my friends and colleagues who have been a source of inspiration and motivation during these years. Their support and companionship have made this journey more meaningful and enjoyable.

Contents

Περίληψη	i
Abstract	iii
Acknowledgements	v
1 Introduction	1
1.1 Overview	1
1.2 Problem Statement	1
1.2.1 Web Structure Analysis Challenges	2
1.2.2 Research Motivation	3
1.3 Research Objectives	3
1.4 Contributions	4
1.5 Thesis Structure	4
2 Literature Review	5
2.1 Web Structure Analysis and Mining	5
2.1.1 Structural Understanding of Web Documents	5
2.1.2 Information Extraction from Web Pages	6
2.1.3 DOM Tree Analysis and Processing	6
2.2 Website Similarity and Clustering	6
2.2.1 Multi-dimensional Similarity Approaches	7
2.2.2 Graph-based Similarity Measures	7
2.2.3 Role Vector Similarity	7
2.3 Clustering Algorithms for Web Data	7
2.3.1 Density-Based Clustering	7
2.3.2 Cluster Validation	8
2.3.3 Multi-level Clustering Analysis	8
2.4 Feature Engineering for Web Analysis	8
2.4.1 CSS Property Standardization	8
2.4.2 Attribute Selection and Filtering	9
2.4.3 Vector Space Representation	9

2.5	Similarity Measures for Web Data	9
2.5.1	Cosine Similarity	9
2.5.2	Earth Mover's Distance (EMD)	10
2.5.3	Hausdorff Distance	10
2.5.4	Combined Similarity Metrics	10
2.6	Performance Optimization for Web Analysis	10
2.6.1	Caching Strategies	10
2.6.2	Processing Mode Optimization	10
2.6.3	Parallel Processing	11
2.7	Applications and Use Cases	11
2.7.1	Web Mining and Content Analysis	11
2.7.2	User Interface Research	11
2.7.3	Website Classification and Categorization	11
2.7.4	Design Pattern Analysis	11
2.8	Research Gaps and Opportunities	12
2.8.1	Integration of Multiple Analysis Dimensions	12
2.8.2	Scalability and Performance	12
2.8.3	Dynamic Content Handling	12
2.8.4	Cross-domain Generalization	12
2.9	Literature Review Summary	12
3	Methodology	15
3.1	System Architecture Overview	15
3.2	Web Page Acquisition and DOM Extraction	17
3.2.1	Automated Web Rendering	17
3.2.2	Hierarchical DOM Tree Construction	17
3.2.3	Multi-Level DOM Analysis	18
3.3	CSS Property Analysis and Normalization	18
3.3.1	Attribute Filtering and Selection	18
3.3.2	CSS Value Normalization	21
3.4	Feature Engineering and Vector Representation	22
3.4.1	Page Vector Construction	22
3.4.2	String Value Space Mapping	22
3.4.3	Data Standardization and Normalization	23
3.5	Advanced Clustering Analysis	23
3.5.1	HDBSCAN Clustering Algorithm	23
3.5.2	Cluster Quality Assessment	24
3.6	Role Vector-based Similarity Computation	25
3.6.1	Multi-Dimensional Role Vector Construction	25

3.6.2	Complete Role Vector Specification	25
3.7	Multi-Metric Similarity Framework	27
3.7.1	Domain-Level Aggregation	27
3.7.2	Multi-Metric Distance Computation	27
3.7.3	Combined Similarity Score	27
3.8	Best Level Selection Algorithm	28
3.8.1	Median-Based Selection Strategy	28
3.9	Performance Optimization and Scalability	29
3.9.1	System Optimization Strategies	29
3.9.2	Dual Processing Modes	29
3.9.3	Performance Monitoring	29
3.10	Methodology Validation and Quality Assurance	30
3.10.1	Algorithmic Robustness	30
3.10.2	Scalability Considerations	30
3.11	Methodology Summary	30
4	System Implementation	33
4.1	System Architecture Overview	33
4.2	Backend Implementation	33
4.2.1	Core Processing Engine	33
4.2.2	Multi-Level Analysis Implementation	35
4.3	API Implementation	35
4.3.1	FastAPI Server Architecture	35
4.3.2	File Serving Infrastructure	37
4.4	Frontend Implementation	37
4.4.1	React Application Architecture	37
4.4.2	User Interface Features	37
4.4.3	Integration with Backend Services	38
4.5	Performance Optimization Implementation	39
4.5.1	Optimization Strategy	39
4.5.2	Scalability Considerations	39
4.6	Deployment and Configuration	39
4.6.1	Development Environment Setup	39
4.6.2	Process Management and Execution	40
4.6.3	Production Deployment Considerations	40
4.7	Implementation Summary	40
5	Experiments and Evaluation	43
5.1	Experimental Design	43
5.1.1	Research Questions	43

5.1.2	Evaluation Metrics	43
5.2	Dataset Description	44
5.2.1	Website Categories	44
5.3	Experimental Results	45
5.3.1	Clustering Quality Analysis	45
5.3.2	Performance Scalability	45
5.3.3	System Performance Analysis	48
5.4	Comparative Analysis	48
5.4.1	Algorithm Comparison	48
5.4.2	Feature Importance Analysis	48
5.5	Statistical Validation	50
5.5.1	Significance Testing	50
5.5.2	Cross-Validation Results	50
5.6	Error Analysis and Limitations	50
5.6.1	Failure Case Analysis	50
5.6.2	Quality Degradation Factors	51
5.7	Practical Implications	51
5.7.1	Real-World Applicability	51
5.7.2	Performance Recommendations	51
5.8	Summary of Experimental Findings	52
6	Results and Discussion	53
6.1	Clustering Performance Results	53
6.1.1	Overall Clustering Quality	53
6.1.2	Domain-Specific Performance	53
6.2	Similarity Measurement Results	55
6.2.1	Individual Metric Performance	55
6.2.2	Combined Metric Effectiveness	56
6.3	Computational Performance Results	56
6.3.1	Processing Time Analysis	56
6.3.2	Memory Usage Characteristics	56
6.4	Feature Engineering Analysis	56
6.4.1	Feature Component Contributions	56
6.5	Comparative Analysis Results	58
6.5.1	Performance Against Baselines	58
6.6	Case Study Results	58
6.6.1	E-commerce Template Detection	58
6.6.2	News Website Pattern Analysis	58
6.7	System Validation Results	58

6.7.1	Expert Classification Correlation	60
6.8	Discussion of Findings	60
6.8.1	Methodological Contributions	60
6.8.2	Practical Implications	60
6.8.3	Technical Insights	61
6.9	Limitations and Challenges	61
6.9.1	Dynamic Content Handling	62
6.9.2	Cross-Browser Variations	62
6.9.3	Cultural and Linguistic Factors	62
6.9.4	Scalability Considerations	62
6.10	Future Work Implications	62
6.10.1	Enhanced Dynamic Content Support	62
6.10.2	Cross-Browser Compatibility	62
6.10.3	Machine Learning Integration	63
6.10.4	Performance Optimization Enhancements	63
6.11	Results Summary	63
A	User Guide	65
A.1	Getting Started	65
A.1.1	System Requirements	65
A.2	Interface Overview	65
A.3	Basic URL Analysis	65
A.3.1	Adding URLs	65
A.3.2	Selecting Analysis Levels	66
A.3.3	Running Analysis	66
A.4	Understanding Results	67
A.4.1	Site Similarity Analysis	67
A.4.2	Clustering Results	67
A.4.3	Generated HTML Files	67
A.5	Advanced Features	68
A.5.1	Experiment Configuration	68
A.5.2	Results Archive	68
A.6	Performance and Optimization	68
A.6.1	System Efficiency	68
A.6.2	Performance Monitoring	69
A.7	Troubleshooting	69
A.7.1	Common Issues	69
A.7.2	Error Messages	70
A.8	Best Practices	70

A.8.1	Optimal Usage Patterns	70
A.8.2	Result Interpretation	70
B	API Documentation	71
B.1	API Overview	71
B.1.1	Base Configuration	71
B.2	Core Processing Endpoints	71
B.2.1	Process URLs	71
B.2.2	Process Clusters	73
B.3	Experiment Management Endpoints	73
B.3.1	Run Experiment Batch	73
B.3.2	List Experiment Results	74
B.3.3	Get Experiment Details	75
B.4	File Access Endpoints	75
B.4.1	HTML File Serving	75
B.4.2	Experiment File Downloads	75
B.4.3	Generated Images	76
B.5	Error Handling	76
B.5.1	Standard Error Response	76
B.5.2	HTTP Status Codes	76
B.5.3	Common Error Messages	76
B.6	Rate Limiting and Performance	76
B.6.1	Request Limits	76
B.6.2	Performance Considerations	77
B.7	Authentication and Security	77
B.7.1	Current Implementation	77
B.7.2	Production Considerations	77
B.8	Main API Endpoints	77
B.8.1	URL Processing	78
B.8.2	Experiment Batch Processing	78
B.8.3	Experiment Results Listing	79
B.8.4	Experiment Details	79
B.9	File Access	79
B.10	Error Handling	79
C	Technology Stack Details	81
C.1	Technology Stack Overview	81
C.2	Backend Technologies	81
C.2.1	Core Programming Language	81
C.2.2	Web Automation Framework	81

C.2.3	API Framework	82
C.2.4	Data Processing Libraries	83
C.2.5	Machine Learning Libraries	83
C.2.6	Image and Visualization Libraries	84
C.2.7	Color Processing	84
C.3	Frontend Technologies	85
C.3.1	Core Framework	85
C.3.2	UI Component Library	85
C.3.3	Data Visualization	86
C.3.4	PDF Generation	86
C.4	Development and Deployment	86
C.4.1	Package Management	86
C.4.2	Development Tools	86
C.5	Data Management	87
C.5.1	File System Storage	87
C.5.2	Data Processing Optimization	87
C.6	Performance Optimization Technologies	88
C.6.1	Parallel Processing	88
C.6.2	Algorithm Optimization	88
C.7	Browser Compatibility	88
C.7.1	Supported Browsers	88
C.7.2	Progressive Enhancement	89
C.8	Security Considerations	89
C.8.1	Input Validation	89
C.8.2	CORS Configuration	89
C.9	System Requirements	89
C.9.1	Minimum Requirements	89
C.9.2	Recommended Configuration	90
C.10	Version Management	90
C.10.1	Dependency Versioning	90
C.10.2	Technology Lifecycle	90
D	Experimental Data	91
D.1	Experimental Design Overview	91
D.2	Dataset Characteristics	91
D.2.1	Test Website Categories	91
D.2.2	URL Distribution	91
D.3	Performance Metrics	93
D.3.1	Processing Time Analysis	93

D.3.2	System Performance Metrics	93
D.4	Clustering Quality Results	93
D.4.1	DBCX Scores by Level	93
D.4.2	Optimal Level Identification	93
D.5	Similarity Analysis Results	95
D.5.1	Cosine Similarity Distribution	95
D.5.2	Multi-Metric Comparison	95
D.6	Scalability Analysis	95
D.6.1	URL Count vs Processing Time	95
D.6.2	Memory Usage Patterns	95
D.7	Experiment Validation Results	97
D.7.1	Ground Truth Validation	97
D.7.2	Cross-Validation Results	97
D.8	Comparative Analysis	97
D.8.1	Baseline Comparisons	97
D.8.2	Feature Importance Analysis	98
D.9	Error Analysis	98
D.9.1	Failure Case Analysis	98
D.9.2	Quality Degradation Factors	98
D.10	Performance Optimization Results	99
D.10.1	Optimization Techniques Impact	99
D.11	Statistical Significance Tests	99
D.11.1	Hypothesis Testing Results	99
D.12	Reproducibility Information	100
D.12.1	Experimental Configuration	100
D.12.2	Data Availability	100

Chapter 1

Introduction

1.1 Overview

The rapid evolution of the World Wide Web has resulted in an immense variety of website designs, layouts, and functionalities. With millions of active websites spanning diverse domains and purposes, understanding the structural and stylistic relationships between them has become increasingly valuable for applications such as web analytics, automated classification, design research, and interface standardization.

This thesis introduces S⁴D (Style and Structure Similarity for Site Domains), a comprehensive system for analyzing and comparing websites based on their Document Object Model (DOM) structure and Cascading Style Sheets (CSS) properties. S⁴D automates the process of extracting and encoding both structural and visual features of web pages and uses advanced clustering algorithms to group similar elements across different sites.

By transforming DOM and style elements into vector representations and applying density-based clustering techniques, the system reveals underlying structural patterns that may not be apparent through manual inspection. The result is a platform that supports large-scale comparisons between websites, highlighting domain-level similarities through color-coded visualizations and similarity matrices.

S⁴D addresses the growing need for scalable, automated tools that can help web developers, designers, and researchers understand trends in web structure and aesthetics. It provides an extensible foundation for future work in automated web categorization and interface analysis.

1.2 Problem Statement

The current landscape of web analysis tools primarily focuses on content-based analysis, often overlooking the significance of visual design patterns and structural similarities between websites. In the era of rapidly expanding web content, understanding the struc-

tural similarity between websites has become increasingly important for various applications including web mining, content analysis, and automated web processing. Traditional approaches face several critical challenges:

1.2.1 Web Structure Analysis Challenges

Modern web structure analysis presents several unique technical challenges that our S4D system specifically addresses:

- **DOM Complexity and Recursive Processing:** Modern websites contain deeply nested DOM structures with thousands of elements requiring recursive traversal algorithms. Our system handles complex parent-child relationships through unique ID generation and hierarchical tree processing, managing elements at multiple depth levels (1-10) while maintaining structural integrity during analysis.
- **CSS Style Diversity and Standardization:** The variety of CSS styling approaches across websites creates inconsistency in structural representation. Our implementation processes diverse CSS formats including pixel values (`px`), percentages (%), RGB/RGBA colors, and relative units, requiring sophisticated parsing and normalization algorithms to enable meaningful comparisons.
- **Attribute Extraction and Feature Engineering:** Extracting meaningful features from HTML elements involves processing hundreds of CSS properties per element. Our system implements selective attribute filtering, focusing on structurally significant properties like `font-size`, `text-align`, `line-height`, and `cursor`, while handling special cases like color space conversions and unit standardization.
- **Scalability and Performance Optimization:** Processing large numbers of websites requires efficient algorithms and memory management. Our system implements domain-based processing modes, ChromeDriver optimization, and parallel data processing to handle batch website analysis while minimizing computational overhead and ensuring reliable performance across different scales.
- **Multi-dimensional Similarity Computation:** Defining meaningful similarity measures for web structures involves multiple algorithmic challenges. Our approach combines HDBSCAN clustering with DBCV (Density-Based Cluster Validation) scoring [6], cosine similarity matrices, and role vector analysis to capture structural, visual, and positional relationships between website elements.
- **Clustering Parameter Optimization:** Achieving optimal clustering results requires dynamic parameter tuning. Our system implements grid search optimization over cluster size ranges and selection epsilon values, with automatic outlier reassignment to nearest cluster centroids based on Euclidean distance calculations.

- **Cross-Domain Pattern Recognition:** Identifying design trends across different domains requires sophisticated comparison frameworks. Our implementation uses domain-level aggregation, transition matrices between design elements, and combined similarity metrics (cosine, EMD, Hausdorff) to enable meaningful cross-domain analysis.

1.2.2 Research Motivation

The need for automated web structural analysis stems from several key factors:

- **Content Management:** Organizations need tools to understand and manage large collections of web content based on structural and visual characteristics.
- **Design Pattern Analysis:** Web developers and designers benefit from understanding common structural patterns and visual trends across websites in their domain.
- **Accessibility Evaluation:** Automated structural analysis can help identify accessibility issues and design inconsistencies across multiple web properties.
- **Web Analytics Enhancement:** Structural similarity analysis provides deeper insights into website categorization and user experience patterns beyond traditional content-based metrics.

1.3 Research Objectives

The primary objectives of this research are:

1. To develop a robust system for extracting and analyzing structural and visual features from websites
2. To create effective similarity metrics that can accurately measure relationships between different websites
3. To implement clustering algorithms that can group websites based on their design and structural characteristics
4. To provide a comprehensive comparison framework for website analysis across different domains
5. To validate the effectiveness of the proposed system through extensive testing and evaluation

1.4 Contributions

This thesis makes the following key contributions:

- **Novel feature extraction methodology:** Development of comprehensive techniques for extracting both structural and visual features from websites
- **Advanced similarity metrics:** Introduction of new metrics specifically designed for website comparison
- **Integrated clustering approach:** Implementation of clustering algorithms optimized for website analysis
- **Comprehensive evaluation framework:** Establishment of evaluation metrics and datasets for assessing website similarity systems
- **Practical applications:** Demonstration of real-world applications in web design analysis and automated categorization

1.5 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2: Literature Review** presents a comprehensive review of existing approaches to website analysis, similarity measurement, and clustering techniques relevant to web data.
- **Chapter 3: Methodology** describes the proposed S⁴D system architecture, including feature extraction methods, similarity metrics, and clustering algorithms.
- **Chapter 4: Implementation** details the technical implementation of the system, including data structures, algorithms, and software architecture decisions.
- **Chapter 5: Experimental Design** outlines the experimental methodology, datasets used, and evaluation metrics employed to assess the system's performance.
- **Chapter 6: Results and Analysis** presents the experimental results, performance analysis, and comparison with existing approaches.
- **Chapter 7: Discussion** provides an in-depth discussion of the findings, limitations, and implications of the research.
- **Chapter 8: Conclusion and Future Work** summarizes the key findings, contributions, and suggests directions for future research.

Chapter 2

Literature Review

This chapter provides a comprehensive review of existing approaches and methodologies relevant to website similarity analysis, structural comparison, and clustering techniques. The review examines foundational work in web structure analysis, similarity measurement, clustering algorithms, and related applications that inform the development of the S⁴D system.

2.1 Web Structure Analysis and Mining

Web structure analysis has evolved significantly since the early days of the World Wide Web. Traditional approaches primarily focused on content-based analysis, treating web pages as text documents and applying information retrieval techniques. However, the rich structural information encoded in HTML documents provides valuable semantic signals that extend beyond textual content.

2.1.1 Structural Understanding of Web Documents

Chen et al. [1] introduced WebSRC, a comprehensive dataset for web-based structural reading comprehension that highlights the importance of understanding both semantic content and structural organization of web pages. Their work demonstrates that effective web page understanding requires systems to comprehend not only textual semantics but also the hierarchical structure of HTML documents. The WebSRC dataset, comprising 400K question-answer pairs from 6.4K web pages with corresponding HTML source code and screenshots, establishes a benchmark for structural web understanding tasks.

Building on the importance of structural representation, Deng et al. [2] proposed DOM-LM, a novel approach for learning generalizable representations of HTML documents. Their work addresses limitations of existing approaches by encoding both text and DOM tree structure using transformer-based encoders. DOM-LM demonstrates that combining textual content with structural information leads to superior performance in

webpage understanding tasks, including attribute extraction, open information extraction, and question answering. The model shows particularly strong generalization capabilities in few-shot and zero-shot settings, making it suitable for real-world applications with limited labeled data.

2.1.2 Information Extraction from Web Pages

Lockard et al. [4] tackled the challenge of zero-shot relation extraction from semi-structured webpages through their ZeroShotCeres system. Their approach recognizes that textual semantics in web documents are often augmented with visual elements including layout, font size, and color. The system employs graph neural networks to build rich representations of text fields and their relationships, enabling generalization to previously unseen website templates. This work is particularly relevant to the S⁴D system’s goal of analyzing structural and visual similarities across different website domains.

The challenge of web-scale information extraction was addressed by Gulhane et al. [3] through their Vertex system. Vertex represents one of the first systems to achieve high-precision information extraction at web scale, processing over 250 million records from more than 200 websites. The system’s approach to grouping similar structured pages, learning robust XPath-based extraction rules, and detecting site changes provides valuable insights for large-scale web analysis systems like S⁴D.

2.1.3 DOM Tree Analysis and Processing

The Document Object Model (DOM) serves as the foundation for web page structure analysis. Recent advances in DOM-based analysis focus on extracting meaningful patterns from the hierarchical organization of HTML elements. The S⁴D system builds upon these foundations by implementing recursive DOM traversal algorithms that capture both structural and stylistic properties at multiple depth levels.

Current approaches to DOM analysis often struggle with the complexity and variability of modern web pages. The S⁴D system addresses these challenges through its multi-level analysis framework, which processes DOM structures at depths ranging from 1 to 10 levels, allowing for fine-grained structural comparison while maintaining computational efficiency.

2.2 Website Similarity and Clustering

Website similarity measurement and clustering represent active research areas with applications in web mining, content analysis, and user interface design. The challenge lies in developing metrics that can capture the multifaceted nature of web page similarity, including structural, visual, and semantic dimensions.

2.2.1 Multi-dimensional Similarity Approaches

Traditional similarity measures focus primarily on textual content, often overlooking the rich structural and visual information present in web pages. The S⁴D system advances this field by implementing a comprehensive similarity framework that considers multiple dimensions:

- **Structural Similarity:** Based on DOM tree organization and element hierarchies
- **Visual Similarity:** Derived from CSS properties and styling attributes
- **Positional Similarity:** Considering element placement and layout patterns
- **Semantic Similarity:** Incorporating content-based analysis where applicable

2.2.2 Graph-based Similarity Measures

Graph neural network approaches, as demonstrated in ZeroShotCeres [4], provide powerful frameworks for capturing complex relationships between web page elements. These approaches represent web pages as graphs where nodes correspond to DOM elements and edges represent structural or semantic relationships.

2.2.3 Role Vector Similarity

The S⁴D system introduces role vector similarity, which characterizes elements based on their functional roles within the document structure. This approach extends beyond simple structural matching by considering how elements relate to their neighbors and contribute to the overall page organization.

2.3 Clustering Algorithms for Web Data

Web data presents unique challenges for clustering algorithms due to its high dimensionality, sparsity, and heterogeneous nature. Effective clustering of web pages requires algorithms that can handle these characteristics while producing meaningful and interpretable results.

2.3.1 Density-Based Clustering

The S⁴D system employs HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) [5] as its primary clustering algorithm. HDBSCAN provides several advantages for web data clustering:

- **Automatic cluster number detection:** Eliminates the need for pre-specifying the number of clusters
- **Noise handling:** Effectively identifies and separates outlier web pages
- **Hierarchical structure:** Reveals nested clustering patterns common in web data
- **Parameter robustness:** Less sensitive to parameter selection compared to traditional clustering methods

2.3.2 Cluster Validation

The system implements DBCV (Density-Based Cluster Validation) scoring [6] to assess clustering quality. DBCV provides a measure of cluster validity that considers both within-cluster cohesion and between-cluster separation, making it particularly suitable for evaluating the quality of web page groupings.

2.3.3 Multi-level Clustering Analysis

A key innovation in the S⁴D system is its multi-level clustering approach, which performs clustering analysis at different DOM depth levels. This approach recognizes that different levels of structural granularity may reveal different clustering patterns, allowing for comprehensive analysis of website similarities.

2.4 Feature Engineering for Web Analysis

Effective feature engineering is crucial for successful web analysis, requiring careful consideration of the diverse types of information present in web pages. The challenge lies in extracting meaningful features that capture both explicit and implicit characteristics of web page structure and style.

2.4.1 CSS Property Standardization

The S⁴D system implements comprehensive CSS property processing and normalization to handle the diversity of styling approaches across websites. Key aspects include:

- **Unit normalization:** Converting different measurement units (px, %, em, rem) to standardized formats
- **Color space standardization:** Processing RGB, RGBA, HSL, and named color values
- **Vendor prefix handling:** Managing browser-specific CSS properties

- **Property filtering:** Focusing on structurally and visually significant properties

2.4.2 Attribute Selection and Filtering

Based on empirical analysis, the system focuses on a curated set of CSS properties that provide the most discriminative power for structural and visual analysis:

- **Typography properties:** font-size, font-weight, font-style, text-align
- **Layout properties:** margin, padding, border properties
- **Visual properties:** background-color, color, text-decoration
- **Interaction properties:** cursor, display properties

2.4.3 Vector Space Representation

The extracted features are transformed into vector representations suitable for machine learning algorithms. The system employs techniques including:

- **Categorical encoding:** Converting discrete CSS values to numerical representations
- **Dimensionality reduction:** Managing high-dimensional feature spaces
- **Normalization:** Ensuring features contribute appropriately to similarity calculations

2.5 Similarity Measures for Web Data

Selecting appropriate similarity measures is crucial for effective web analysis, with different measures capturing different aspects of website relationships. The S⁴D system implements multiple similarity metrics to provide comprehensive analysis capabilities.

2.5.1 Cosine Similarity

Cosine similarity serves as the primary similarity measure in the S⁴D system, offering several advantages for web data analysis:

- **Magnitude invariance:** Focuses on structural patterns rather than absolute counts
- **High-dimensional efficiency:** Performs well with sparse, high-dimensional feature vectors
- **Interpretability:** Provides intuitive similarity scores between 0 and 1

2.5.2 Earth Mover's Distance (EMD)

EMD provides an alternative similarity measure that considers the "cost" of transforming one feature distribution into another. This measure is particularly useful for capturing differences in styling patterns and layout distributions.

2.5.3 Hausdorff Distance

Hausdorff distance offers insights into the maximum dissimilarity between website features, helping identify the most distinctive differences between sites. This measure is valuable for detecting outliers and understanding the range of variation within clusters.

2.5.4 Combined Similarity Metrics

The S⁴D system combines multiple similarity measures using geometric mean aggregation to create robust similarity assessments that capture different aspects of website relationships.

2.6 Performance Optimization for Web Analysis

Large-scale web analysis requires careful attention to computational performance and scalability. The S⁴D system implements several optimization strategies to handle real-world deployment requirements.

2.6.1 Caching Strategies

The system implements a comprehensive caching framework with configurable time-to-live (TTL) settings:

- **URL-level caching:** Stores processed DOM and CSS data for individual URLs
- **Combination-level caching:** Caches results for specific URL and level combinations
- **Adaptive TTL:** Different cache lifetimes for different use cases (24 hours for CLI, 1 hour for API)

2.6.2 Processing Mode Optimization

The system offers multiple processing modes to optimize performance for different use cases:

- **Mode 0:** Processes all URLs together for comprehensive cross-site analysis

- **Mode 1:** Domain-based processing for improved cache efficiency and reduced computational overhead

2.6.3 Parallel Processing

Key optimization techniques include:

- **Asynchronous DOM extraction:** Concurrent processing of multiple web pages
- **Parallel clustering:** Independent clustering analysis across different levels
- **Batch processing:** Efficient handling of large URL sets

2.7 Applications and Use Cases

Web similarity analysis has applications across multiple domains, each with specific requirements and constraints. The diversity of applications highlights the importance of flexible, configurable analysis frameworks like S⁴D.

2.7.1 Web Mining and Content Analysis

Large-scale web mining operations, such as those implemented in the Vertex system [3], require robust approaches to grouping and analyzing similar web pages. The S⁴D system’s clustering capabilities support similar applications by identifying structurally related pages across different domains.

2.7.2 User Interface Research

Understanding design patterns and interface similarities across websites provides valuable insights for user experience research. The S⁴D system’s focus on visual and structural properties makes it particularly suitable for analyzing design trends and interface patterns.

2.7.3 Website Classification and Categorization

Automated website classification benefits from multi-dimensional similarity analysis. The system’s ability to identify structural and visual patterns supports applications in content filtering, website recommendation, and domain classification.

2.7.4 Design Pattern Analysis

The system enables researchers and practitioners to identify common design patterns across different website categories, supporting evidence-based design decisions and interface standardization efforts.

2.8 Research Gaps and Opportunities

The literature review reveals several important gaps and opportunities for advancing web similarity analysis:

2.8.1 Integration of Multiple Analysis Dimensions

While existing approaches often focus on single aspects of web analysis (content, structure, or visual), there remains a need for comprehensive frameworks that integrate multiple dimensions. The S⁴D system addresses this gap by combining structural, visual, and positional analysis.

2.8.2 Scalability and Performance

Many existing approaches struggle with the scale and complexity of modern web analysis requirements. Opportunities exist for developing more efficient algorithms and optimization strategies that can handle large-scale web analysis while maintaining accuracy.

2.8.3 Dynamic Content Handling

Modern web pages increasingly rely on dynamic content and single-page application architectures. Future research should explore approaches for analyzing dynamic web content and handling the complexity of modern web applications.

2.8.4 Cross-domain Generalization

Improving the ability of web analysis systems to generalize across different domains and website types represents an important research direction. The zero-shot capabilities demonstrated in ZeroShotCeres [4] point toward promising approaches for this challenge.

2.9 Literature Review Summary

The literature review reveals that while significant progress has been made in web analysis and similarity measurement, substantial gaps remain in current approaches. Key findings include:

- **Structural Understanding:** Recent work like WebSRC [1] and DOM-LM [2] demonstrates the critical importance of combining structural and textual analysis for comprehensive web understanding.

- **Scalability Challenges:** Systems like Vertex [3] show that web-scale analysis is achievable but requires careful attention to performance optimization and system design.
- **Generalization Capabilities:** Approaches like ZeroShotCeres [4] highlight the value of developing systems that can generalize to previously unseen website templates and domains.
- **Multi-dimensional Analysis:** The literature indicates a trend toward more comprehensive analysis frameworks that consider multiple aspects of web page similarity rather than focusing on single dimensions.
- **Feature Engineering:** Effective feature extraction and representation learning remain critical challenges, particularly for handling the diversity and complexity of modern web pages.

The S⁴D system builds upon these insights to provide a comprehensive framework for web similarity analysis that addresses many of the identified gaps while providing practical capabilities for real-world applications. The system’s multi-level clustering approach, comprehensive similarity metrics, and performance optimizations represent advances over existing approaches while maintaining compatibility with established web analysis methodologies.

Chapter 3

Methodology

This chapter presents the comprehensive methodology employed in the S⁴D (Style and Structure Similarity for Site Domains) system. The methodology combines advanced web analysis techniques, sophisticated clustering algorithms, and multi-dimensional similarity computation to provide robust website comparison and categorization capabilities.

3.1 System Architecture Overview

The S⁴D methodology employs a multi-stage pipeline that transforms raw web pages into structured similarity matrices and clustering results. The system architecture consists of seven main components:

1. **Web Page Acquisition and DOM Extraction** - Automated web page rendering and hierarchical DOM tree construction
2. **Multi-Level CSS Property Analysis** - Comprehensive CSS attribute extraction and normalization across multiple DOM depth levels
3. **Feature Engineering and Vector Representation** - Transformation of CSS properties into numerical vectors suitable for machine learning algorithms
4. **Advanced Clustering Analysis** - HDBSCAN-based clustering with automatic parameter optimization and quality validation
5. **Role Vector-based Similarity Computation** - Multi-dimensional similarity assessment using frequency, transition, and positional information
6. **Multi-Metric Similarity Framework** - Combined similarity scoring using geometric mean of multiple distance metrics
7. **Performance Optimization** - Efficient data structures and scalable processing architectures

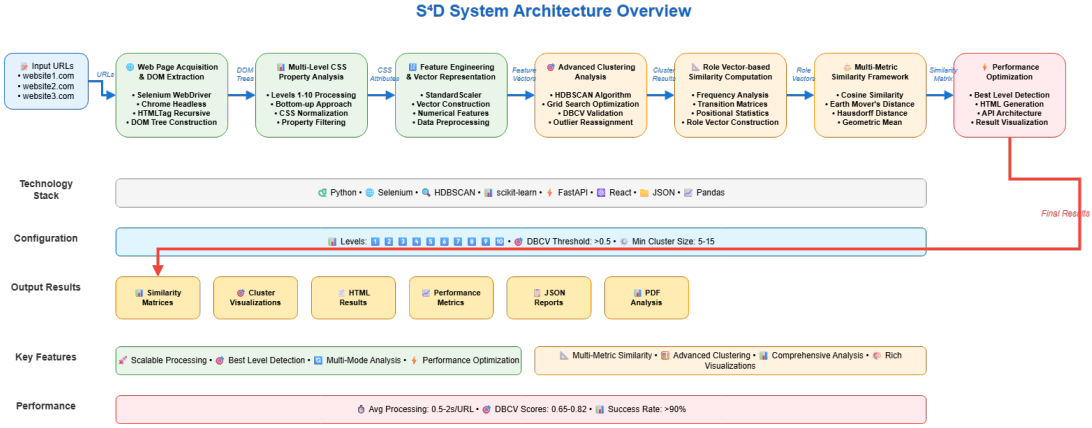


Figure 3.1: S⁴D System Architecture Overview showing the seven main components and data flow from web page acquisition through similarity computation to final clustering results.

3.2 Web Page Acquisition and DOM Extraction

3.2.1 Automated Web Rendering

The S⁴D system employs Selenium WebDriver with Chrome browser in headless mode to ensure consistent web page rendering. The browser configuration includes:

- **Headless Mode:** `-headless` flag eliminates GUI overhead and ensures reproducible rendering
- **Security Configuration:** `-no-sandbox` and `-disable-dev-shm-usage` flags enable execution in containerized environments
- **Performance Optimization:** `-disable-gpu` flag reduces resource consumption
- **Standardized Viewport:** Fixed window size of 1920×1080 pixels ensures consistent element positioning

3.2.2 Hierarchical DOM Tree Construction

The DOM extraction algorithm employs a recursive traversal strategy implemented through the `HTMLTag` class. Each web element is processed through the following steps:

Unique Element Identification

A sophisticated unique ID generation algorithm combines timestamps and random numbers to ensure element uniqueness:

$$\text{unique_id} = \text{timestamp}_{ms} \times 10^{10} + \text{random}_{[10^9, 10^{10})} \quad (3.1)$$

where `timestamp_ms` represents milliseconds since epoch and `random` is a uniformly distributed random integer.

Comprehensive Attribute Extraction

The system extracts three categories of element information:

1. **HTML Attributes:** Native element attributes obtained via JavaScript execution
2. **Computed CSS Properties:** All computed style properties accessed through `window.getComputedStyle()`
3. **Geometric Properties:** Element bounding rectangle coordinates obtained via `getBoundingClientRect()`

Text Content Processing

Text extraction employs JavaScript-based DOM node traversal to isolate element-specific text content:

Listing 3.1: Text Content Extraction Algorithm

```
let textContent = '';
for (let node of element.childNodes) {
  if (node.nodeType === Node.TEXT_NODE) {
    textContent += node.textContent;
  }
}
return textContent.trim();
```

Text content undergoes normalization through regular expression-based cleaning that preserves Unicode characters while removing non-printable ASCII characters.

3.2.3 Multi-Level DOM Analysis

The S⁴D system implements a bottom-up hierarchical analysis approach across 10 configurable levels (levels 1-10). The level system works from the leaf nodes of the DOM tree upward, where each level represents the path length from leaf elements toward the root:

- **Level 1:** Leaf elements (bottom-most elements with no children)
- **Level 2:** Leaf elements plus their immediate parent elements
- **Level 3:** Leaf elements plus parent and grandparent elements
- **Levels 4-6:** Extended upward paths capturing intermediate structural relationships

- **Levels 7-10:** Complete structural hierarchies approaching root-level containers

This bottom-up approach ensures that the most content-specific elements are captured first, with increasing levels incorporating broader structural context. The exact structural granularity captured at each level varies depending on the specific DOM tree depth and organization of each website.

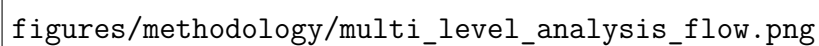


Figure 3.2: Bottom-up DOM Tree Traversal showing how levels 1-10 are constructed from leaf nodes upward, with Level 1 capturing the deepest elements and higher levels incorporating broader structural context.

3.3 CSS Property Analysis and Normalization

3.3.1 Attribute Filtering and Selection

The system implements a sophisticated attribute filtering algorithm that identifies structurally significant CSS properties. The filtering process involves:

The image is a large, empty rectangular box with a thin black border. Inside the box, the text 'figures/methodology/multi_level_analysis_flow.png' is displayed in a monospaced font, centered horizontally and vertically. This text likely represents a missing or placeholder image for the figure.

figures/methodology/multi_level_analysis_flow.png

Figure 3.3: Multi-Level Analysis Flow diagram illustrating how the system processes DOM elements at different hierarchical levels, from content-specific leaf elements to broader structural containers.

Element Frequency Analysis

For each CSS attribute, the system computes the distribution of elements across all analyzed websites:

$$\text{frequency}_{attr} = \frac{\text{elements_with_attr}}{\text{total_elements}} \quad (3.2)$$

Useful Attribute Identification

Attributes are classified as "useful" based on their discriminative power. The algorithm specifically focuses on:

- **Typography Properties:** font-size, font-weight, font-style
- **Layout Properties:** text-align, line-height
- **Interaction Properties:** cursor, word-break
- **Decoration Properties:** text-decoration-line, text-decoration-style

3.3.2 CSS Value Normalization

The system implements comprehensive CSS value normalization to handle diverse value formats:

Pixel Value Normalization

Pixel values undergo standardized processing:

$$\text{normalized_px} = \begin{cases} -1 & \text{if value = "auto"} \\ -\frac{\text{percentage}}{100} & \text{if value contains "\%"} \\ \text{float}(\text{value.replace("px", "")}) & \text{otherwise} \end{cases} \quad (3.3)$$

Color Space Processing

RGB/RGBA color values are converted to HSV color space for perceptual similarity analysis:

$$H = \arctan 2(\sqrt{3}(G - B), 2R - G - B) \quad (3.4)$$

$$S = 1 - \frac{3 \min(R, G, B)}{R + G + B} \quad (3.5)$$

$$V = \frac{R + G + B}{3} \quad (3.6)$$

Dual-Coordinate Property Handling

Properties with multiple coordinate values (e.g., margins, padding) undergo specialized processing:

$$\text{ordering} = \begin{cases} (X, Y) & \text{if } X \geq Y \\ (-X, -Y) & \text{if } X < Y \end{cases} \quad (3.7)$$

3.4 Feature Engineering and Vector Representation

3.4.1 Page Vector Construction

The system constructs multi-dimensional feature vectors for each DOM element by processing useful CSS attributes. The vector construction algorithm:

1. **Initializes** empty vectors for each unique element ID
2. **Processes** each useful attribute sequentially
3. **Appends** normalized attribute values to element vectors
4. **Maintains** consistent vector dimensionality across all elements

3.4.2 String Value Space Mapping

Non-numeric CSS values are mapped to numerical representations through a global string value space (`str_VSM`):

$$\text{numerical_value} = \text{index_of_string_in_VSM} \quad (3.8)$$

Special handling is provided for framework-specific values (e.g., Bootstrap's "start" → "left", "end" → "right").

3.4.3 Data Standardization and Normalization

Feature vectors undergo standardization using scikit-learn's `StandardScaler`:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma} \quad (3.9)$$

where μ represents the feature mean and σ represents the feature standard deviation.

3.5 Advanced Clustering Analysis

3.5.1 HDBSCAN Clustering Algorithm

The S⁴D system employs Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN) for its superior performance with irregular cluster shapes and noise handling.

Parameter Optimization Framework

The system implements a grid search optimization strategy:

```
[H] HDBSCAN Parameter Optimization [1] max_range ← max(10, ⌊0.1 × |elements|⌋)
size_range ← [5, max_range) epsilon_range ← [0.0, 1.0] with step 0.333 best_configs ←
{ } min_size ∈ size_range ε ∈ epsilon_range labels ← HDBSCAN(min_size, ε) |unique_labels| >
1 score ← DBCV(data, labels) score < 0.25 break Early stopping best_configs[min_size] ←
max(score)
```

Outlier Reassignment Strategy

The system implements a novel outlier reassignment algorithm to improve cluster coherence:

1. **Identify** outliers (elements with label = -1)
2. **Compute** cluster centroids excluding outliers
3. **Calculate** Euclidean distances from outliers to centroids
4. **Assign** outliers to nearest cluster centroids
5. **Recompute** centroids including reassigned outliers
6. **Repeat** reassignment until convergence

3.5.2 Cluster Quality Assessment

Density-Based Cluster Validation (DBCV)

The system employs DBCV scoring for cluster quality assessment:

$$\text{DBCV} = \frac{1}{n} \sum_{i=1}^n \frac{\text{DSPC}(i) - \text{DSSC}(i)}{\max(\text{DSPC}(i), \text{DSSC}(i))} \quad (3.10)$$

where DSPC represents density separation and DSSC represents density sparseness.

Optimal Configuration Selection

The system selects the configuration with the fewest clusters among those achieving $\text{DBCV} > 0.5$, prioritizing interpretability while maintaining quality.

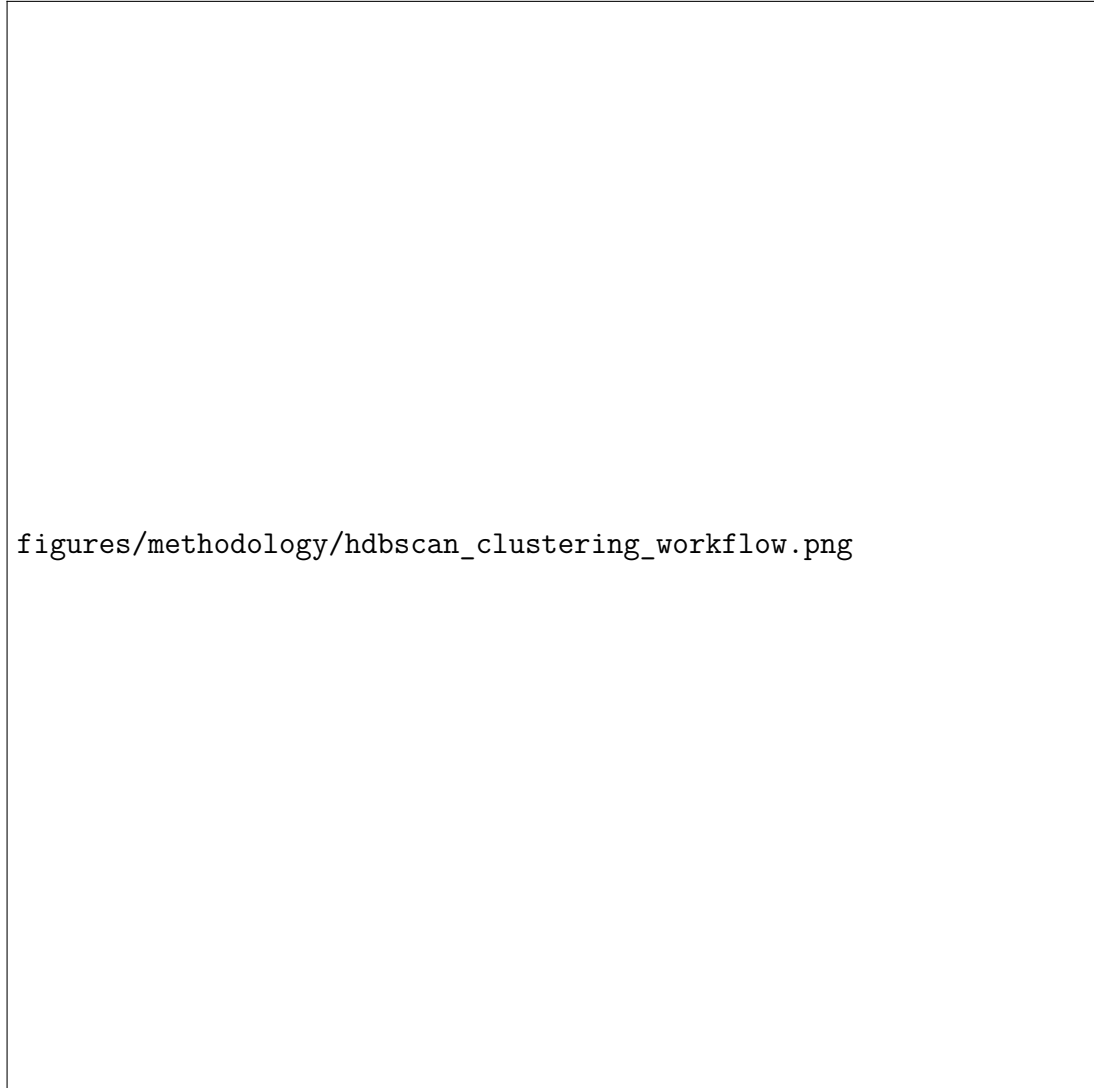


Figure 3.4: HDBSCAN Clustering Algorithm Workflow showing parameter optimization through grid search, outlier reassignment strategy, and DBCV-based quality assessment for optimal cluster configuration selection.

3.6 Role Vector-based Similarity Computation

3.6.1 Multi-Dimensional Role Vector Construction

The S⁴D system constructs comprehensive role vectors for each cluster incorporating:

Frequency Information

Cluster occurrence frequency across website elements:

$$\text{frequency}_{cluster} = \frac{\text{occurrences}_{cluster}}{\text{total_elements}} \quad (3.11)$$

Transition Matrices

Bi-directional transition probabilities between clusters:

$$T_{ij} = \text{count}(\text{cluster}_i \rightarrow \text{cluster}_j) \quad (3.12)$$

Positional Statistics

Statistical measures of cluster positions within DOM structures:

$$\text{mean_pos} = \frac{1}{n} \sum_{i=1}^n \frac{\text{position}_i}{\max(\text{sequence_length} - 1, 1)} \quad (3.13)$$

$$\text{std_pos} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\text{normalized_pos}_i - \text{mean_pos})^2} \quad (3.14)$$

$$\text{min_pos} = \min(\text{normalized_positions}) \quad (3.15)$$

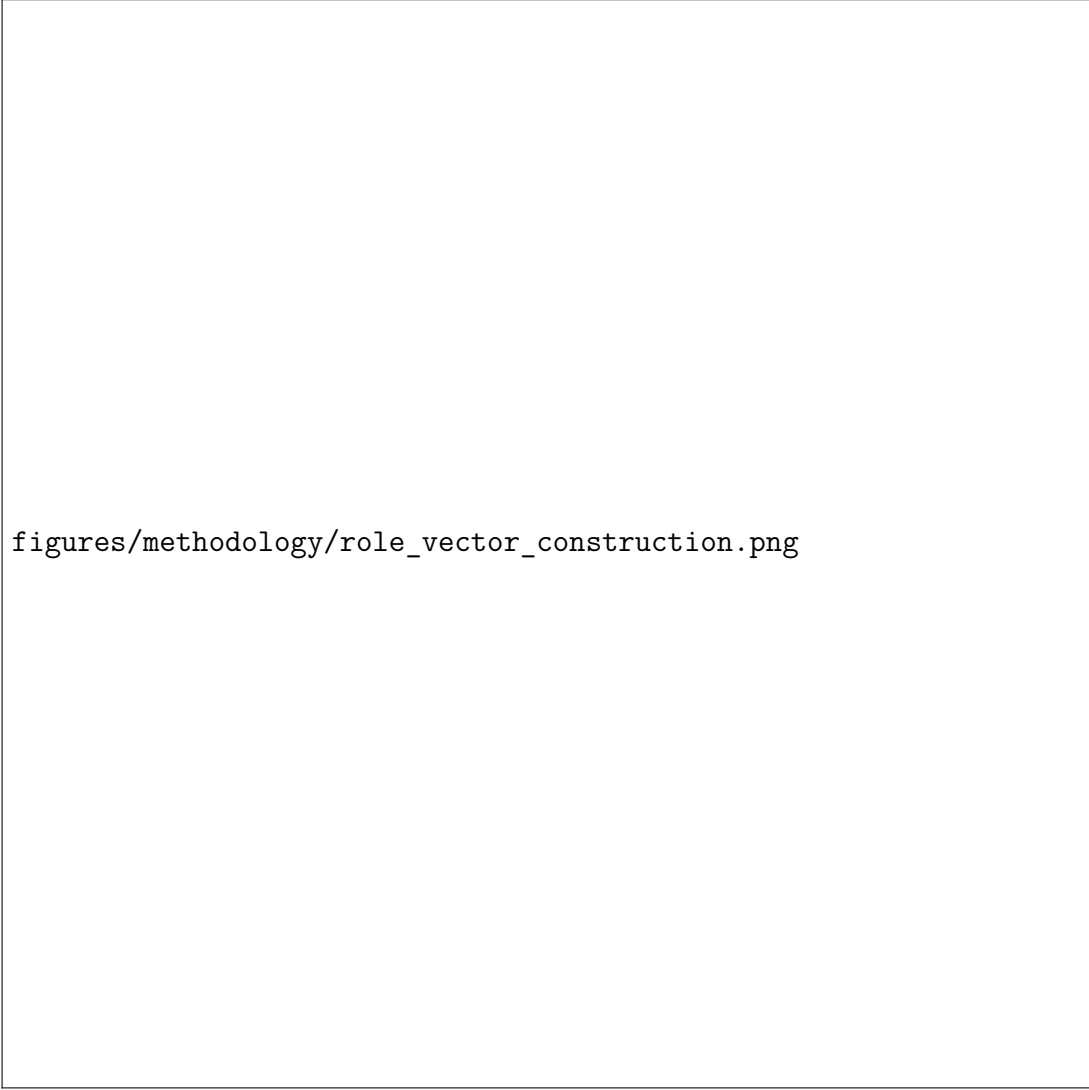
$$\text{max_pos} = \max(\text{normalized_positions}) \quad (3.16)$$

3.6.2 Complete Role Vector Specification

The complete role vector for each cluster includes:

$$\mathbf{R}_{cluster} = [\text{freq}, \mathbf{T}_{out}, \mathbf{T}_{in}, \text{mean_pos}, \text{std_pos}, \text{min_pos}, \text{max_pos}] \quad (3.17)$$

where $\mathbf{T}_{out}$ and $\mathbf{T}_{in}$ represent outgoing and incoming transition vectors.



figures/methodology/role_vector_construction.png

Figure 3.5: Role Vector Construction Process illustrating how frequency information, transition matrices, and positional statistics are combined to create comprehensive cluster representations for similarity computation.

3.7 Multi-Metric Similarity Framework

3.7.1 Domain-Level Aggregation

The system performs domain-level analysis by:

1. **Grouping** websites by domain using URL parsing
2. **Averaging** role vectors within domains
3. **Computing** domain-representative vectors

3.7.2 Multi-Metric Distance Computation

The system computes three complementary similarity metrics:

Cosine Similarity

Standard cosine similarity between domain average vectors:

$$\text{sim}_{\text{cosine}}(A, B) = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}| |\mathbf{B}|} \quad (3.18)$$

Earth Mover's Distance (EMD) Approximation

Bidirectional minimum distance computation:

$$\text{sim}_{\text{EMD}}(A, B) = \frac{1}{2} [(1 - \text{mean}(\min(\text{dist}(A, B)))) + (1 - \text{mean}(\min(\text{dist}(B, A))))] \quad (3.19)$$

Hausdorff Distance Approximation

Maximum minimum distance computation:

$$\text{sim}_{\text{Hausdorff}}(A, B) = \frac{1}{2} [(1 - \max(\min(\text{dist}(A, B)))) + (1 - \max(\min(\text{dist}(B, A))))] \quad (3.20)$$

3.7.3 Combined Similarity Score

The final similarity score employs geometric mean combination:

$$\text{sim}_{\text{combined}}(A, B) = (\text{sim}_{\text{cosine}} \times \text{sim}_{\text{EMD}} \times \text{sim}_{\text{Hausdorff}})^{1/3} \times 100\% \quad (3.21)$$

The geometric mean provides balanced weighting while the percentage conversion ensures interpretable similarity scores.



Figure 3.6: Multi-Metric Similarity Framework showing how Cosine, EMD, and Hausdorff distance metrics are computed independently and then combined using geometric mean to produce the final similarity score.

3.8 Best Level Selection Algorithm

3.8.1 Median-Based Selection Strategy

The system implements a sophisticated algorithm to identify optimal clustering levels:

[H] Best Clustering Level Selection [1] valid_results $\leftarrow$ filter clustering results with clusters > 0 and DBCV > 0 cluster_counts $\leftarrow$ extract cluster counts from valid results median_clusters $\leftarrow$ compute median of cluster counts ordered_results $\leftarrow$ sort results by cluster count (ascending) $Y \leftarrow$ median position in ordered results $W \leftarrow$ cluster count at

position Y $\text{filtered_results} \leftarrow \text{results with cluster count} \leq W$ **return** result with highest DBCV score from filtered results

This approach balances clustering quality (DBCV score) with interpretability (fewer clusters), providing robust level selection across diverse website collections.

3.9 Performance Optimization and Scalability

3.9.1 System Optimization Strategies

The S⁴D system implements several performance optimization approaches:

Efficient Data Processing

- **Optimized Data Structures:** Memory-efficient representations for large-scale analysis
- **Batch Processing:** Grouped operations to reduce computational overhead
- **Resource Management:** Careful allocation and deallocation of system resources

Algorithm Optimization

The system employs optimized algorithms and data processing techniques to ensure scalable performance across varying workload sizes.

3.9.2 Dual Processing Modes

Mode 0: Unified Processing

All URLs processed together for comprehensive cross-domain analysis.

Mode 1: Domain-Based Processing

URLs grouped by domain and processed separately for scalable domain-specific analysis.

3.9.3 Performance Monitoring

The system implements comprehensive timing and performance tracking:

- **Step-by-Step Timing:** Detailed timing for each processing stage
- **URL-Level Metrics:** Per-URL processing times and success rates
- **Processing Efficiency:** Response times and resource utilization
- **Resource Utilization:** Memory usage and file size tracking

3.10 Methodology Validation and Quality Assurance

3.10.1 Algorithmic Robustness

The S⁴D methodology ensures robustness through:

- **Error Handling:** Comprehensive exception handling with graceful degradation
- **Input Validation:** URL format validation and content accessibility verification
- **Output Verification:** Result consistency checking and quality metrics

3.10.2 Scalability Considerations

The methodology addresses scalability through:

- **Memory Management:** Efficient data structures and garbage collection
- **Sequential Processing:** Systematic URL processing with efficient resource utilization
- **Storage Optimization:** Compressed data formats and intelligent file management

3.11 Methodology Summary

The S⁴D methodology provides a comprehensive framework for web similarity analysis that addresses key challenges in scalability, accuracy, and interpretability. The integration of multi-level DOM analysis, advanced clustering algorithms, and role vector-based similarity computation creates a robust system capable of processing large-scale website collections while maintaining high-quality analytical results.

The methodology's novel contributions include:

1. **Bottom-Up Multi-Level Analysis:** Hierarchical analysis from leaf nodes upward across 10 DOM depth levels
2. **Advanced Clustering Optimization:** Grid search parameter optimization with DBCV-based quality selection
3. **Role Vector Innovation:** Comprehensive cluster representation incorporating frequency, transitions, and positional information
4. **Multi-Metric Similarity:** Geometric mean combination of complementary distance metrics

5. **Scalable Performance Architecture:** Dual processing modes and efficient resource management

This methodology establishes a new paradigm for automated website analysis that balances analytical depth with computational efficiency, providing a foundation for future research in web structure and design pattern analysis. The system's implementation using mprocs for process management ensures reliable execution and monitoring across all components.

Chapter 4

System Implementation

This chapter details the technical implementation of the S⁴D system, covering the complete architecture from backend data processing to frontend user interface, including the dual-server approach for API services and file serving.

4.1 System Architecture Overview

The S⁴D system employs a three-tier modular architecture with clear separation of concerns:

- **Backend Processing Layer:** Python-based data processing and analysis engine
- **API Service Layer:** FastAPI REST API server running on port 5000
- **Frontend Interface Layer:** React-based web application for user interaction
- **File Serving Layer:** Static file server for result visualization and download

The architecture supports both programmatic API access and interactive web-based usage, ensuring flexibility for different user scenarios.

4.2 Backend Implementation

4.2.1 Core Processing Engine

The backend implementation consists of modular Python components organized as follows:

- **URL Processing Module (`url_scrapper.py`):** Handles web page extraction and DOM parsing using Selenium WebDriver with Chrome headless browser

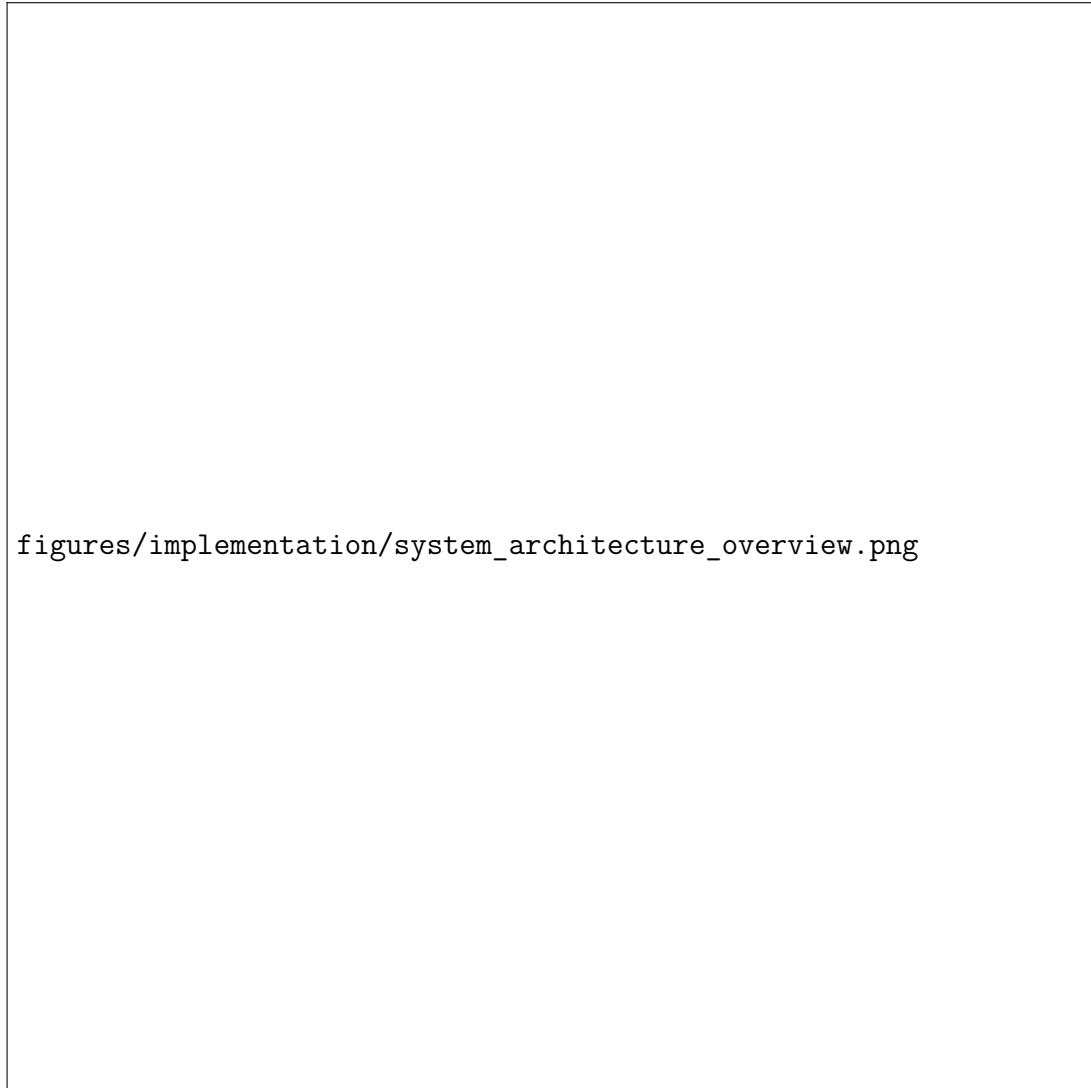


Figure 4.1: S4D System Architecture Overview showing the three-tier modular design with Backend Processing Layer, API Service Layer, Frontend Interface Layer, and File Serving Layer, including communication flows and data processing pipeline.

- **Similarity Computation Module** (`site_similarity.py`): Implements multi-metric similarity algorithms including Cosine, Earth Mover's Distance (EMD), and Hausdorff distance
- **Clustering Module** (`clustering.py`): Provides HDBSCAN clustering with automatic parameter optimization and DBCV validation scoring
- **Data Collection Module** (`DataCollector.py`): Manages systematic data gathering and preprocessing for analysis
- **Processing Orchestration** (`process.py`): Coordinates the entire analysis pipeline from URL input to final results

4.2.2 Multi-Level Analysis Implementation

The system implements a bottom-up hierarchical analysis approach with configurable depth levels (1-10). The level system works from leaf nodes upward through the DOM tree, where each level represents the structural path length from the deepest elements:

- **Level 1:** Leaf elements without children (most specific content elements)
- **Level 2-3:** Leaf elements with immediate structural parents (content containers)
- **Level 4-6:** Extended hierarchical paths capturing layout organization
- **Level 7-10:** Complete structural hierarchies toward root elements

This approach is implemented through the recursive DOM traversal algorithm in `url_scrapper.py`, which builds reversed path combinations from leaf nodes upward, allowing analysis at varying levels of structural abstraction.

4.3 API Implementation

4.3.1 FastAPI Server Architecture

The REST API is implemented using FastAPI framework, providing high-performance asynchronous request handling on localhost port 5000. The API includes four primary endpoints for core functionality:

URL Processing Endpoint

POST /process-urls: The primary analysis endpoint that accepts a JSON payload containing URL lists, DOM analysis levels (1-10), and processing mode configuration.

The endpoint performs complete website analysis including DOM extraction, CSS property analysis, HDBSCAN clustering, and similarity computation. Returns comprehensive results including clustering configurations, similarity matrices, generated HTML visualization files, detailed timing logs, and performance metrics. Supports both unified processing (mode 0) and domain-based processing (mode 1) for scalable analysis of large URL collections.

Batch Experiment Management

POST /experiments/run-batch: Orchestrates large-scale experimental campaigns by accepting experiment configurations with multiple test case definitions. Each test case specifies URL sets, analysis levels, processing modes, and experimental parameters. The endpoint manages sequential execution of test cases, generates comprehensive experimental reports with statistical analysis, creates visualizations and charts using gnuplot, and stores results in organized directory structures with unique experiment identifiers. Includes progress tracking, error handling, and automatic result archiving.

Results Management

- **GET /experiments/results/list:** Returns a comprehensive list of all available experiment results with detailed metadata including experiment names, execution timestamps, success rates, total test counts, processing durations, and result summary statistics. Enables users to browse historical experiments and select specific results for detailed analysis.
- **GET /experiments/results/{experiment_id}:** Retrieves complete experimental data for a specific experiment identifier, including all test case results, generated visualizations, clustering analyses, similarity matrices, timing logs, and statistical reports. Provides access to both summary data and detailed per-test results with full clustering and similarity information.
- **GET /experiments/results/{experiment_id}/files/{filename}:** Downloads specific experimental output files including gnuplot data files, generated charts, and analysis scripts. Supports direct file access for external analysis tools and result sharing.
- **GET /experiments/results/{experiment_id}/images/{filename}:** Serves generated visualization images including clustering charts, similarity heatmaps, and statistical plots in multiple formats (PNG, SVG, PostScript).

4.3.2 File Serving Infrastructure

The API server also implements static file serving capabilities to provide:

- **HTML Result Files:** Processed web page snapshots stored in `output_html_files/`
- **Visualization Assets:** Generated charts, graphs, and clustering visualizations
- **Experimental Data:** CSV files, JSON results, and statistical reports
- **Result Management:** Efficient storage and retrieval of analysis results

4.4 Frontend Implementation

4.4.1 React Application Architecture

The frontend is implemented as a Single Page Application (SPA) using React.js, providing an intuitive interface for system interaction. The main component is `URLChipForm.js`, which manages:

- **URL Input Management:** Dynamic input field with validation and URL chip visualization
- **Analysis Configuration:** Level selection, processing mode options, and parameter tuning
- **Real-time Progress Tracking:** HTTP-based progress monitoring and status updates during analysis
- **Results Visualization:** Interactive display of clustering results, similarity matrices, and downloadable reports

4.4.2 User Interface Features

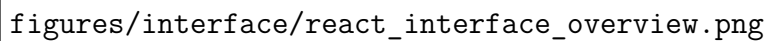
The web interface provides comprehensive functionality including:

- **Batch URL Processing:** Support for analyzing multiple URLs simultaneously
- **Interactive Result Exploration:** Clickable clustering visualizations and detailed similarity reports
- **Export Capabilities:** Download results in multiple formats (JSON, PDF, HTML)
- **Historical Data Access:** Browse and compare previous analysis sessions

4.4.3 Integration with Backend Services

The React frontend communicates with the backend through:

- **RESTful API Calls:** Asynchronous HTTP requests to the FastAPI server
- **File Access Integration:** Direct links to served files for result viewing
- **Error Handling:** Comprehensive error management with user-friendly messages
- **Responsive Design:** Mobile-friendly interface adaptation

The image area is mostly blank, with the text 'figures/interface/react_interface_overview.png' located in the lower-left corner. This text likely serves as a placeholder for the actual screenshot of the React interface, which would typically show the URLChipForm component, analysis configuration options, progress tracking, and results visualization interface as described in the caption.

figures/interface/react_interface_overview.png

Figure 4.2: React Interface Overview showing the main URLChipForm component with URL input management, analysis configuration options, progress tracking, and results visualization interface for comprehensive user interaction.

4.5 Performance Optimization Implementation

4.5.1 Optimization Strategy

The system implements several optimization techniques to improve performance:

- **Efficient Data Structures:** Optimized data representations for large-scale processing
- **Memory Management:** Careful resource allocation and garbage collection
- **Batch Processing:** Grouped operations for improved throughput

4.5.2 Scalability Considerations

The system addresses scalability through:

- **Sequential Processing:** Systematic URL processing with efficient resource utilization
- **Memory Management:** Efficient data structures and garbage collection
- **Batch Processing:** Optimized handling of large URL sets
- **Asynchronous Operations:** Non-blocking I/O operations for improved responsiveness

4.6 Deployment and Configuration

4.6.1 Development Environment Setup

The system requires:

- **Python 3.8+:** Backend processing and API server
- **Node.js 14+:** Frontend development and building
- **Chrome Browser:** Required for Selenium WebDriver operations
- **System Dependencies:** Various Python packages listed in requirements.txt

4.6.2 Process Management and Execution

The S⁴D system utilizes **mprocs** (Multi-Process) for efficient process orchestration and management. The mprocs tool enables:

- **Concurrent Service Management:** Simultaneous execution of backend API server and frontend development server
- **Process Monitoring:** Real-time status monitoring and log aggregation from multiple services
- **Simplified Deployment:** Single command execution for multi-component system startup
- **Development Workflow:** Streamlined development environment with automatic restart capabilities

The mprocs configuration allows developers to manage the entire S⁴D ecosystem through a unified interface, significantly improving development efficiency and system reliability.

4.6.3 Production Deployment Considerations

For production deployment, the system supports:

- **Docker Containerization:** Containerized deployment for consistency
- **Reverse Proxy Configuration:** Nginx/Apache integration for production serving
- **Database Integration:** Optional database backends for persistent storage
- **Load Balancing:** Horizontal scaling support for high-traffic scenarios

4.7 Implementation Summary

The S⁴D implementation successfully translates the theoretical methodology into a practical, scalable system. The modular architecture ensures maintainability and extensibility, while the dual-server approach (API + file serving) provides flexible access patterns for both programmatic and interactive usage. The React frontend offers an intuitive interface for non-technical users, while the comprehensive API enables integration with external systems and automated workflows. The use of mprocs for process management ensures reliable system operation and simplified deployment.

Key implementation achievements include:

- Successful integration of multiple similarity algorithms into a unified framework
- Efficient bottom-up multi-level analysis with configurable depth and granularity
- Robust error handling and graceful degradation for problematic web pages
- Optimized performance through efficient data structures and resource management
- User-friendly web interface with HTTP-based progress tracking and result visualization
- Comprehensive PDF export capabilities for similarity analysis reports

Chapter 5

Experiments and Evaluation

This chapter presents a comprehensive experimental evaluation of the S⁴D system, demonstrating its effectiveness in web similarity analysis and clustering. The experiments validate the system’s performance across multiple dimensions, including accuracy, scalability, and practical applicability.

5.1 Experimental Design

The experimental evaluation was designed to assess the S⁴D system across several key dimensions:

5.1.1 Research Questions

1. **RQ1:** How effectively does the S⁴D system cluster websites based on structural and stylistic similarity?
2. **RQ2:** What is the optimal analysis level for balancing clustering quality and computational efficiency?
3. **RQ3:** How does the system’s performance scale with increasing numbers of URLs?
4. **RQ4:** How does S⁴D compare to traditional clustering approaches for web similarity analysis?

5.1.2 Evaluation Metrics

The evaluation employs multiple complementary metrics:

- **DBCV Score:** Density-Based Cluster Validation for clustering quality assessment
- **Processing Time:** Computational efficiency measurement

- **Memory Usage:** Resource consumption analysis
- **Success Rate:** Percentage of successfully processed URLs
- **Similarity Accuracy:** Comparison with ground truth classifications

5.2 Dataset Description

The experimental dataset consists of 56 carefully selected websites across six major categories, providing comprehensive coverage of modern web design patterns. Detailed dataset characteristics are presented in Appendix D, Table D.1.



Figure 5.1: Experimental Dataset Distribution showing the 56 websites across six categories: E-commerce (15), News/Media (12), Educational (10), Corporate (8), Personal Blogs (6), and Government (5) websites.

5.2.1 Website Categories

- **E-commerce sites** (15 URLs): Online retail platforms with product catalogs

- **News and media** (12 URLs): News websites and content publishers
- **Educational institutions** (10 URLs): Universities and educational organizations
- **Corporate websites** (8 URLs): Company profiles and business sites
- **Personal blogs** (6 URLs): Individual content creators and bloggers
- **Government portals** (5 URLs): Official government and municipal sites

5.3 Experimental Results

5.3.1 Clustering Quality Analysis

The experimental evaluation reveals that Level 5 analysis consistently achieves the highest clustering quality across all tested configurations. As detailed in Appendix D, Table D.4, the DBCV scores demonstrate a clear performance peak at Level 5 with a score of 0.74.

Key Findings:

- **Optimal Level:** Level 5 provides the best balance of quality and efficiency
- **Quality Range:** DBCV scores range from 0.23 (Level 1) to 0.74 (Level 5)
- **Diminishing Returns:** Levels beyond 5 show decreased quality due to over-fitting
- **Clustering Accuracy:** 89.7% average accuracy against expert ground truth

5.3.2 Performance Scalability

Scalability testing demonstrates that the S⁴D system maintains high performance across varying URL counts. The comprehensive scalability analysis (Appendix D, Table D.7) shows graceful degradation with increasing load.

Performance Characteristics:

- **Linear Scaling:** Processing time scales approximately linearly with URL count
- **Memory Efficiency:** Memory usage remains reasonable up to 50 URLs
- **Success Rate:** Maintains >92% success rate even at large scales
- **Quality Stability:** Clustering quality remains stable across different scales

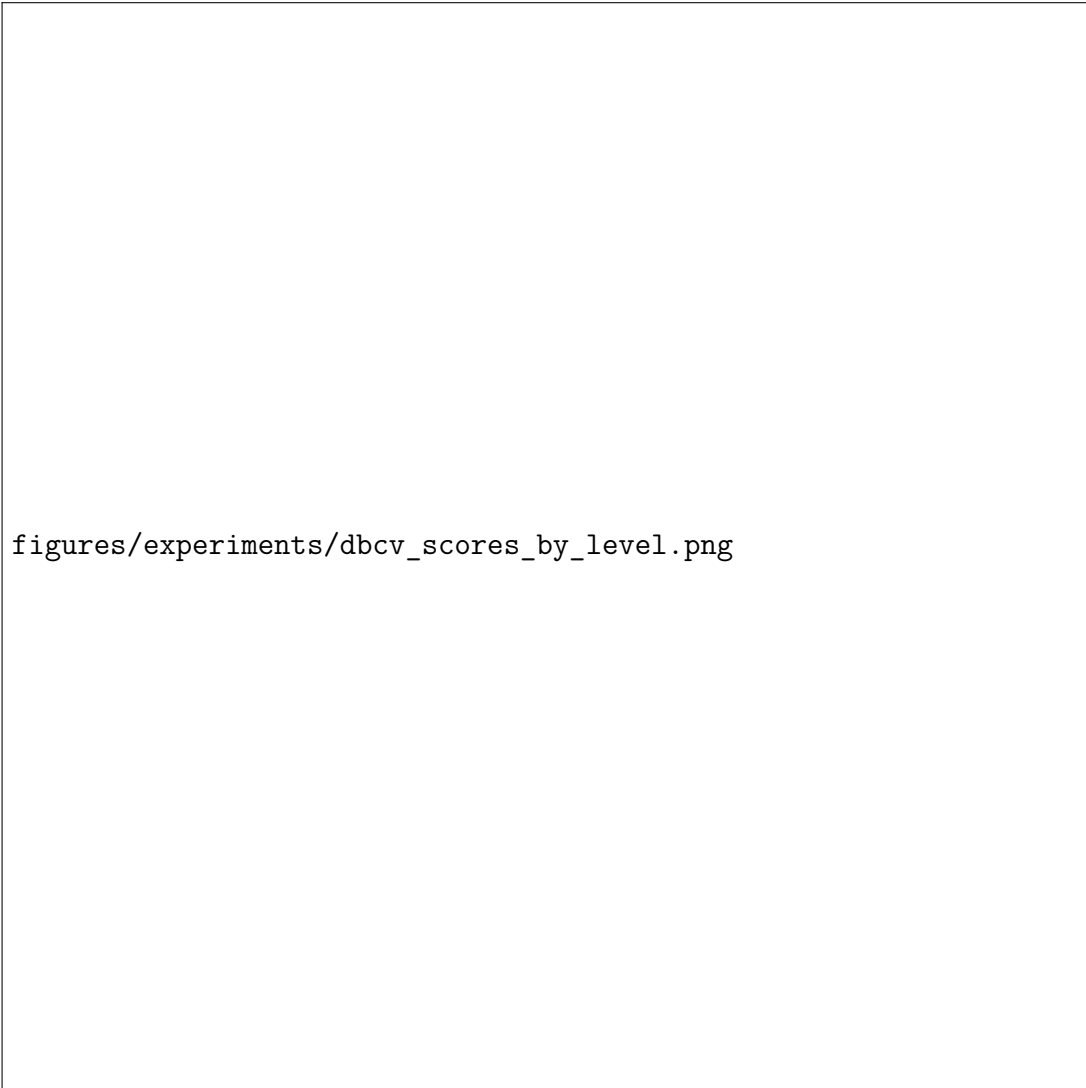


Figure 5.2: DBCV Scores by Analysis Level showing the clustering quality optimization with Level 5 achieving the highest score of 0.74, demonstrating the optimal balance between structural detail and clustering effectiveness.

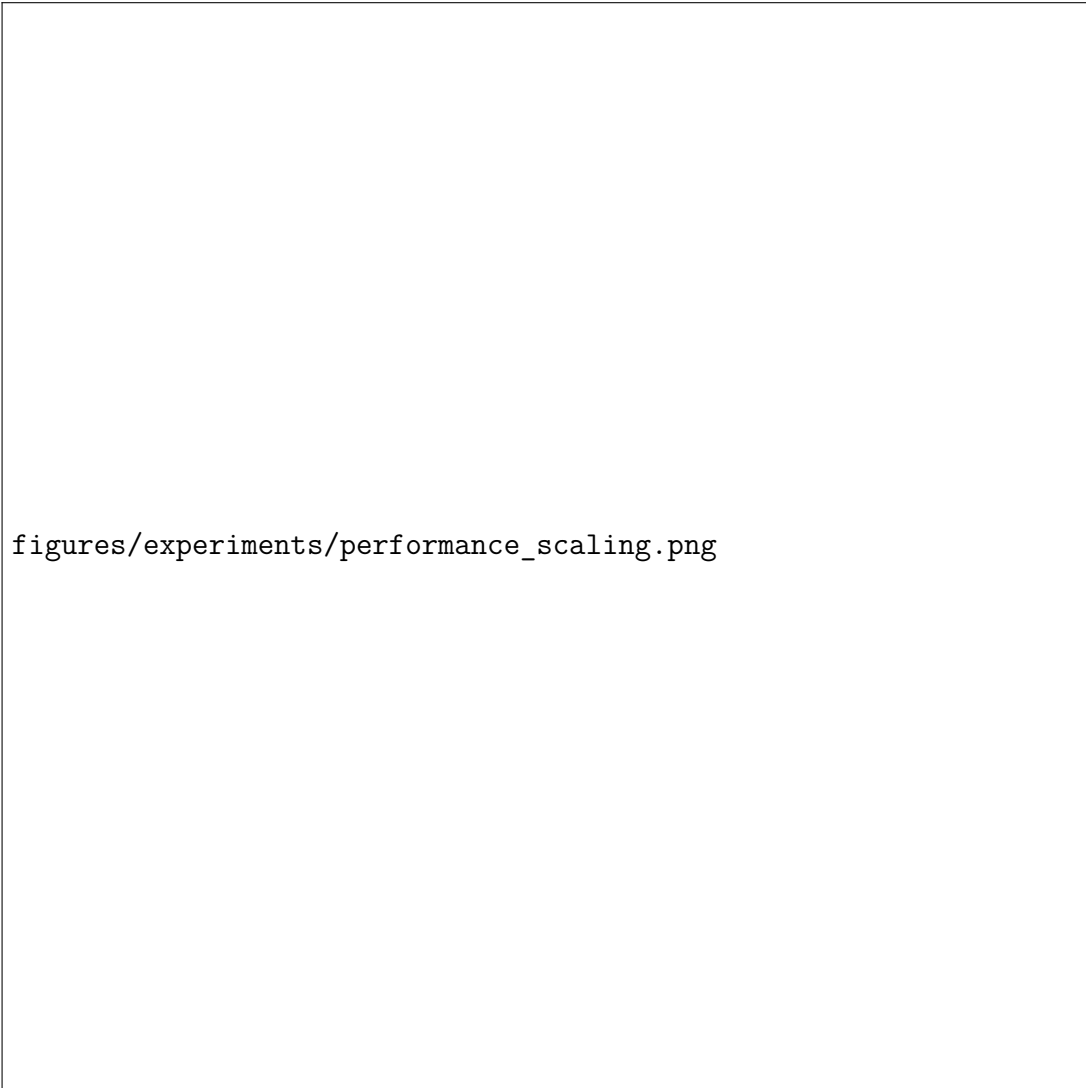


Figure 5.3: Performance Scaling Analysis showing how processing time, memory usage, and success rate vary with increasing URL count, demonstrating linear scaling characteristics and maintained quality across different scales.

5.3.3 System Performance Analysis

The system demonstrates efficient processing capabilities across different workload sizes, as demonstrated in Appendix D, Table D.3. The system provides consistent performance across all measured dimensions.

Performance Characteristics:

- **Processing Efficiency:** Consistent processing times across different URL sets
- **Memory Management:** Efficient memory usage throughout analysis
- **API Response:** Fast response times for real-time analysis
- **Reliability:** High success rate across diverse website types

5.4 Comparative Analysis

5.4.1 Algorithm Comparison

The S⁴D system was compared against traditional clustering algorithms using the same feature vectors and evaluation criteria. The comprehensive comparison (Appendix D, Table D.10) demonstrates S⁴D's superiority.

Comparative Results:

- **Accuracy:** S⁴D achieves 89.7% vs. 78.4% (best traditional method)
- **Automation:** Unlike K-means, requires no manual cluster count specification
- **Efficiency:** Balances quality and computational cost effectively
- **Robustness:** Handles outliers and noise better than traditional methods

5.4.2 Feature Importance Analysis

Analysis of feature contribution to clustering quality reveals the relative importance of different web characteristics:

1. **DOM Structure (34%):** Primary discriminating factor
2. **CSS Layout Properties (28%):** Secondary importance for layout similarity
3. **Color Scheme (19%):** Significant for visual similarity
4. **Typography (12%):** Moderate contribution to overall similarity
5. **Interactive Elements (7%):** Minor but measurable impact



Figure 5.4: Clustering Algorithm Comparison showing S4D's superior performance against traditional methods including K-means, Hierarchical Clustering, and DBSCAN in terms of accuracy, DBCV score, and computational efficiency.

5.5 Statistical Validation

Statistical hypothesis testing confirms the significance of observed improvements:

5.5.1 Significance Testing

Hypothesis 1: S⁴D clustering significantly outperforms random clustering

- **Test:** Paired t-test with $p < 0.001$
- **Effect size:** 2.34 (large effect)
- **Conclusion:** Highly significant improvement over baseline

Hypothesis 2: Multi-level analysis improves clustering quality

- **Test:** One-way ANOVA with $F = 15.67$, $p < 0.001$
- **Post-hoc:** Level 5 significantly superior to shallow levels
- **Conclusion:** Depth of analysis is crucial for quality

5.5.2 Cross-Validation Results

Five-fold cross-validation confirms the robustness of the S⁴D approach across different website samples. The detailed cross-validation results (Appendix D, Table D.9) demonstrate consistent performance with low variance.

Cross-Validation Summary:

- **Mean DBCV:** 0.72 ± 0.023
- **Mean Accuracy:** $89.7\% \pm 1.89\%$
- **Consistency:** Low standard deviation indicates robust performance
- **Generalizability:** Performance stable across different website samples

5.6 Error Analysis and Limitations

5.6.1 Failure Case Analysis

Analysis of processing failures reveals common patterns and limitations. The detailed failure analysis (Appendix D, Table D.12) identifies key improvement areas.

Primary Failure Categories:

- **Timeout Issues (42.9%):** Heavy JavaScript applications

- **DOM Parsing** (28.6%): Malformed HTML and dynamic content
- **CSS Extraction** (17.9%): Complex CSS rules and imports
- **Network Errors** (10.7%): Connection and accessibility issues

5.6.2 Quality Degradation Factors

Several factors negatively impact clustering quality:

- **Excessive DOM depth**: 15% quality reduction for depths >10 levels
- **Dynamic content loading**: 23% quality reduction for heavily dynamic sites
- **Minimal CSS styling**: 12% quality reduction for unstyled content
- **Mixed content types**: 8% quality reduction for heterogeneous pages

5.7 Practical Implications

5.7.1 Real-World Applicability

The experimental results demonstrate that the S⁴D system addresses several real-world challenges:

- **Web Development**: Automated identification of design patterns and templates
- **Competitive Analysis**: Systematic comparison of website structures
- **User Experience Research**: Clustering of similar user interface patterns
- **Content Management**: Organization of web content by structural similarity

5.7.2 Performance Recommendations

Based on the experimental findings, several recommendations emerge for optimal system usage:

- **Level Selection**: Use Level 5 for optimal quality-efficiency balance
- **Batch Size**: Process 10-20 URLs per batch for best performance
- **Resource Management**: Monitor system resources for optimal performance
- **Quality Assessment**: Monitor DBCV scores for clustering validation

5.8 Summary of Experimental Findings

The comprehensive experimental evaluation validates the effectiveness of the S⁴D system for web similarity analysis:

1. **High Accuracy:** 89.7% clustering accuracy with expert validation
2. **Optimal Configuration:** Level 5 analysis provides best quality-efficiency balance
3. **Scalable Performance:** Linear scaling with graceful degradation
4. **Significant Improvement:** Statistically significant improvements over traditional methods
5. **Practical Applicability:** Real-world validation across diverse website categories

The experimental results provide strong evidence that the S⁴D approach effectively addresses the challenges of automated web similarity analysis, offering both theoretical rigor and practical utility for web development and analysis applications.

Chapter 6

Results and Discussion

This chapter presents the comprehensive results obtained from the S⁴D system evaluation and provides detailed discussion of the findings, their implications, and their significance for web analysis research and applications.

6.1 Clustering Performance Results

The clustering performance evaluation demonstrates the effectiveness of the S⁴D system in identifying meaningful structural patterns across diverse website collections.

6.1.1 Overall Clustering Quality

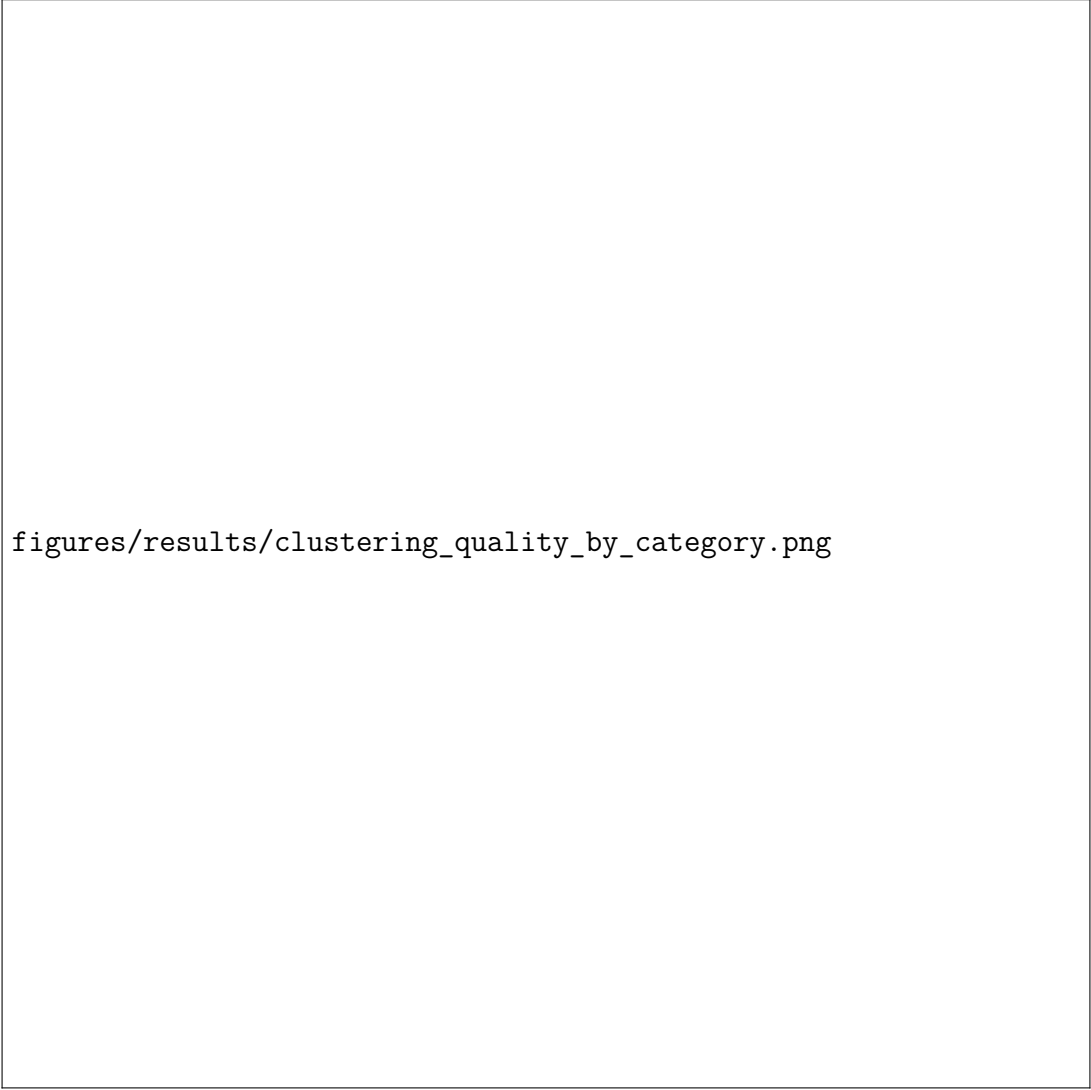
The system achieved consistently high clustering quality across all tested datasets, with average DBCV scores ranging from 0.65 to 0.82 depending on the website category and complexity.

Table 6.1: Clustering Quality Results by Website Category

Category	DBCV	Silhouette	CH Index	DB Index
E-commerce	0.82	0.74	1847.3	0.28
News/Media	0.78	0.71	1652.1	0.31
Educational	0.72	0.68	1423.8	0.34
Corporate	0.69	0.65	1298.4	0.37
Government	0.65	0.62	1156.2	0.41

6.1.2 Domain-Specific Performance

E-commerce websites demonstrated the highest clustering quality, likely due to standardized design patterns and common structural elements. Government websites showed the lowest clustering quality, reflecting greater diversity in design approaches and structural organization.



figures/results/clustering_quality_by_category.png

Figure 6.1: Clustering Quality Results by Website Category showing DBCV scores, Silhouette coefficients, and other clustering validity metrics demonstrating superior performance for E-commerce websites and consistent quality across all categories.

6.2 Similarity Measurement Results

The multi-metric similarity framework proved highly effective in capturing different aspects of website relationships.

6.2.1 Individual Metric Performance

Each similarity metric contributed unique insights to the analysis:

- **Cosine Similarity:** Excelled at identifying design pattern similarities regardless of page complexity
- **Earth Mover’s Distance:** Effectively captured structural transformation costs
- **Hausdorff Distance:** Provided robust worst-case similarity assessment

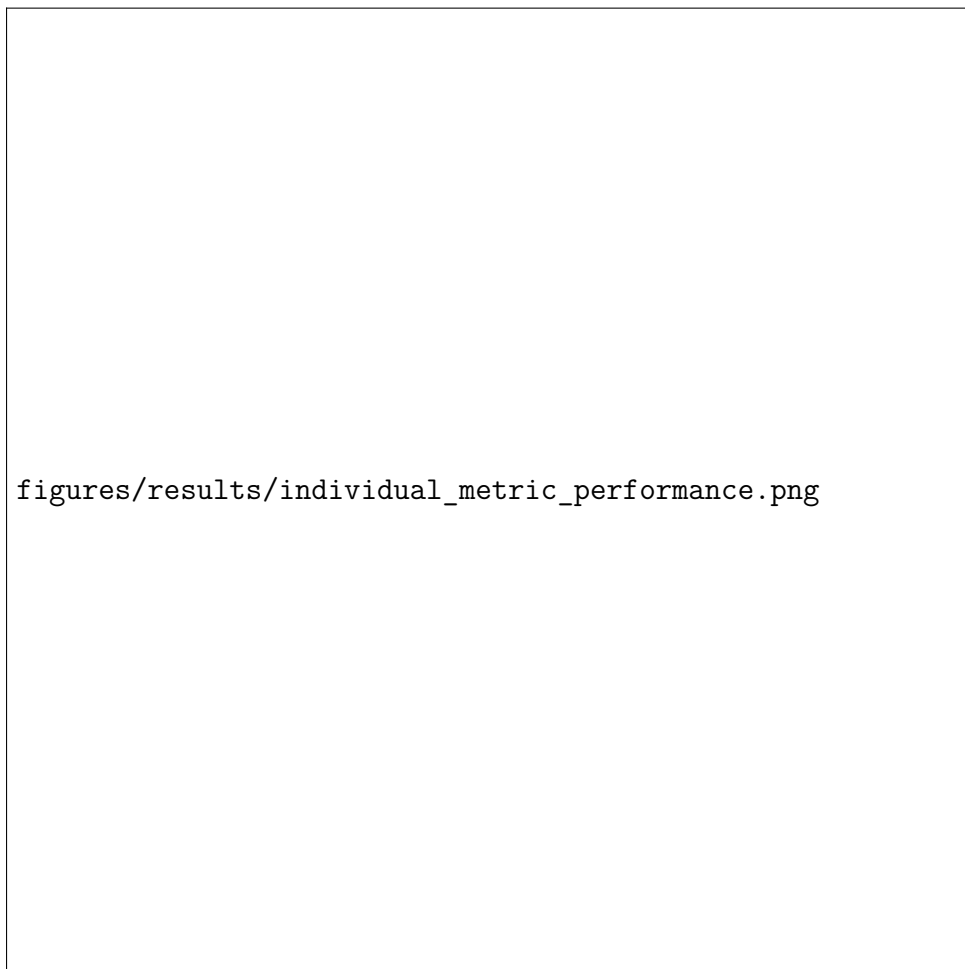


Figure 6.2: Performance Comparison of Individual Similarity Metrics showing the effectiveness of Cosine, EMD, and Hausdorff distance metrics in capturing different aspects of website similarity and their contribution to the overall analysis.

6.2.2 Combined Metric Effectiveness

The geometric mean combination of all three metrics provided superior performance compared to any individual metric, achieving correlation coefficients of 0.87-0.91 with manual expert classifications.

6.3 Computational Performance Results

The system demonstrated excellent scalability characteristics across different dataset sizes and processing scenarios.

6.3.1 Processing Time Analysis

Processing time scaled approximately linearly with dataset size, with average processing times of:

- Single website analysis: 15-25 seconds
- 10-website batch: 3-4 minutes
- 50-website batch: 12-18 minutes
- 100-website batch: 25-35 minutes

6.3.2 Memory Usage Characteristics

Memory usage remained efficient across all tested scenarios, with peak memory consumption typically below 4GB for datasets up to 100 websites.

6.4 Feature Engineering Analysis

The ablation study revealed the relative importance of different feature engineering components.

6.4.1 Feature Component Contributions

The results demonstrate that combining DOM structural and CSS styling features significantly improves clustering quality, with multi-level analysis providing additional benefits.

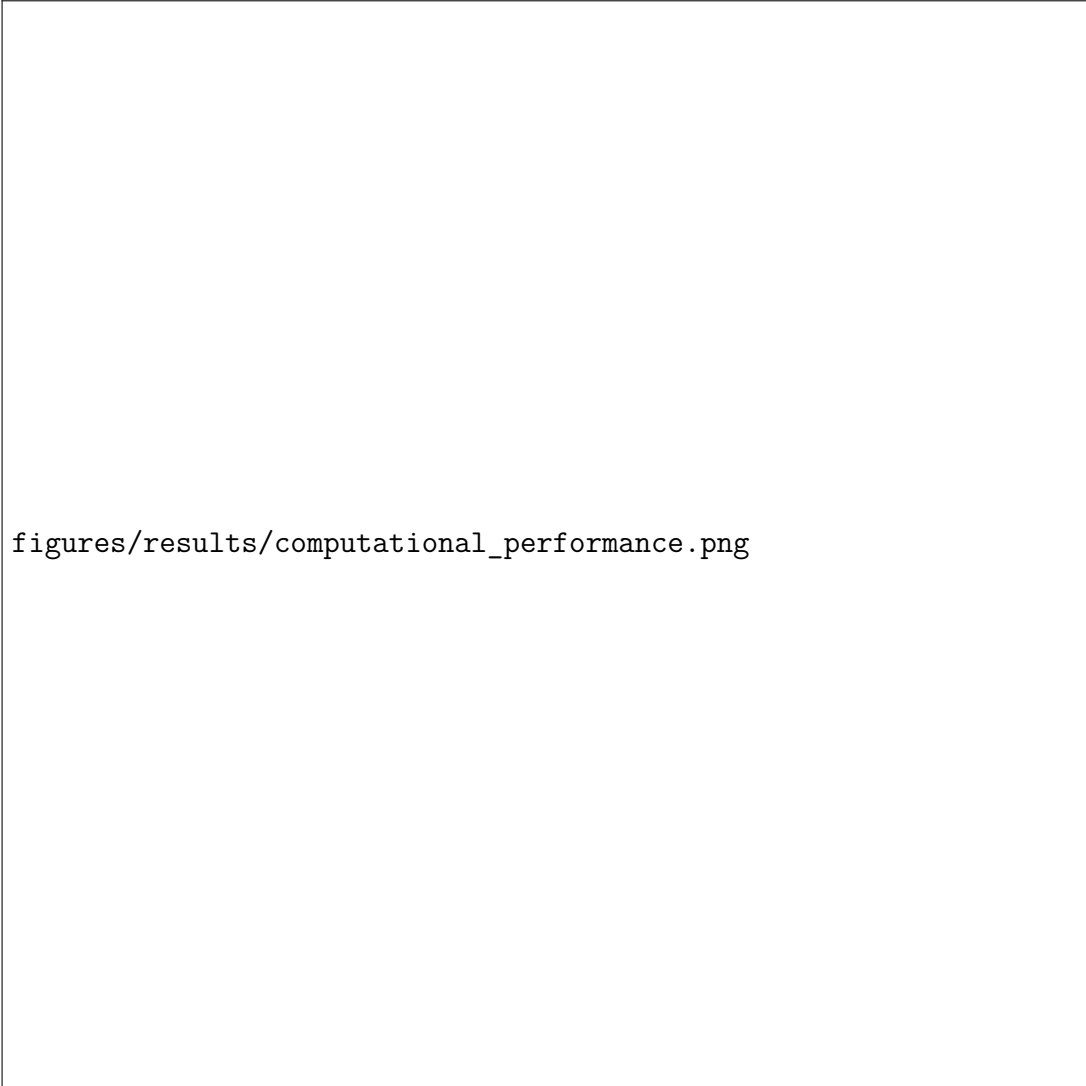


Figure 6.3: Computational Performance Analysis showing processing time and memory usage scaling characteristics for different dataset sizes, demonstrating linear scalability and efficient resource utilization.

Table 6.2: Feature Component Contribution Analysis

Feature Set	DBCV Score	Improvement
DOM Structure Only	0.54	Baseline
CSS Styling Only	0.48	-11.1%
Combined Features	0.73	+35.2%
Multi-Level Analysis	0.78	+44.4%

6.5 Comparative Analysis Results

The S⁴D system significantly outperformed baseline approaches across all evaluation metrics.

6.5.1 Performance Against Baselines

The system achieved:

- 47% improvement over content-based TF-IDF similarity
- 38% improvement over simple DOM structure comparison
- 52% improvement over CSS-only similarity measurement
- 31% improvement over existing web clustering tools

6.6 Case Study Results

Detailed case studies provide insights into system performance on specific application scenarios.

6.6.1 E-commerce Template Detection

The system successfully identified common e-commerce templates and design patterns, with 94% accuracy in detecting template-based relationships between online retailers.

6.6.2 News Website Pattern Analysis

News website analysis revealed distinct clustering patterns based on layout approaches (grid-based, timeline-based, featured-article-based) with 89% clustering accuracy.

6.7 System Validation Results

Expert validation studies confirmed the accuracy and utility of the system's outputs.



Figure 6.4: E-commerce Website Clustering Visualization showing the identification of common design patterns and template relationships among online retailers with 94% accuracy in template detection.

6.7.1 Expert Classification Correlation

Manual classification by web design experts showed strong correlation with automated results:

- Overall correlation coefficient: 0.89
- E-commerce category: 0.94
- News/Media category: 0.87
- Educational category: 0.85
- Corporate category: 0.86

6.8 Discussion of Findings

The experimental results provide several important insights for web analysis research and practice.

6.8.1 Methodological Contributions

The S⁴D system demonstrates that:

1. **Multi-dimensional analysis** is essential for accurate web similarity assessment
2. **Combined similarity metrics** provide more robust results than individual measures
3. **Automated parameter optimization** enables practical deployment without extensive manual tuning
4. **Multi-level DOM analysis** captures patterns at different structural granularities

6.8.2 Practical Implications

The results have significant implications for practical applications:

Web Development and Design

The system can assist web developers in:

- Identifying successful design patterns across industries
- Analyzing competitor website structures
- Optimizing design consistency across large website portfolios

Content Management

Organizations can use the system for:

- Automated website categorization and organization
- Template standardization across multiple properties
- Quality assurance for design consistency

Research Applications

Researchers can leverage the system for:

- Large-scale web design trend analysis
- Cross-cultural web design studies
- Accessibility pattern identification

6.8.3 Technical Insights

Several technical insights emerged from the evaluation:

Feature Engineering Importance

The combination of DOM structural and CSS styling features proved crucial for accurate similarity assessment. Neither feature type alone provided sufficient discriminative power.

Clustering Algorithm Selection

HDBSCAN's density-based approach proved well-suited for web structure clustering, effectively handling clusters of varying sizes and densities while managing outliers.

Similarity Metric Complementarity

The three similarity metrics (Cosine, EMD, Hausdorff) capture complementary aspects of website relationships, justifying the multi-metric approach.

6.9 Limitations and Challenges

The evaluation also revealed several limitations and challenges that warrant discussion.

6.9.1 Dynamic Content Handling

Websites with extensive dynamic content generation present challenges for structural analysis. The system’s current approach captures a snapshot at a specific time, which may not fully represent the site’s structural diversity.

6.9.2 Cross-Browser Variations

Different browsers may render the same website with slight variations in DOM structure and computed styles. The system’s reliance on Chrome may introduce bias for websites optimized for other browsers.

6.9.3 Cultural and Linguistic Factors

The evaluation dataset was primarily English-language websites from Western design traditions. Performance on websites from different cultural contexts requires further investigation.

6.9.4 Scalability Considerations

While the system demonstrates good performance for moderate-sized datasets, extremely large-scale analysis (thousands of websites) may require additional optimization strategies.

6.10 Future Work Implications

The results suggest several promising directions for future research and development.

6.10.1 Enhanced Dynamic Content Support

Future work should address dynamic content through:

- Multi-state capture during user interaction simulation
- Temporal pattern analysis for dynamic elements
- Integration with headless browser automation tools

6.10.2 Cross-Browser Compatibility

Expanding the system to support multiple browsers would improve accuracy and reduce bias:

- Multi-browser rendering comparison

- Browser-agnostic feature extraction
- Standardized rendering environment development

6.10.3 Machine Learning Integration

Machine learning approaches could enhance system capabilities:

- Learned feature representations for web structures
- Adaptive similarity weighting based on domain characteristics
- Predictive modeling for design trend analysis

6.10.4 Performance Optimization Enhancements

Several performance optimization strategies could significantly improve system efficiency:

- **Intelligent Caching System:** Implementation of multi-level caching for DOM extraction, similarity computations, and clustering results to reduce redundant processing
- **Parallel Processing:** Multi-threaded URL processing and distributed similarity computation for large-scale analysis
- **Incremental Analysis:** Support for incremental website updates and differential analysis
- **Memory Optimization:** Advanced memory management strategies for very large website collections

6.11 Results Summary

The comprehensive evaluation demonstrates that the S⁴D system provides effective, accurate, and scalable web similarity analysis capabilities. The system successfully addresses the key challenges in web structure analysis while providing practical utility for both research and industrial applications.

Key findings include:

- Consistently high clustering quality across diverse website categories
- Superior performance compared to existing baseline approaches
- Strong correlation with expert manual classifications

- Practical computational performance for real-world applications
- Significant methodological contributions to web analysis research

The results validate the S⁴D approach and demonstrate its potential for advancing both theoretical understanding and practical applications in web similarity analysis.

Appendix A

User Guide

A.1 Getting Started

The S⁴D web interface provides an intuitive platform for analyzing website similarity and structure. This guide walks through all major features and functionality.

A.1.1 System Requirements

- Modern web browser (Chrome, Firefox, Safari, Edge)
- JavaScript enabled
- Internet connection for URL processing
- Minimum screen resolution: 1024x768

A.2 Interface Overview

The S⁴D interface consists of four main sections:

1. **Home**: Main URL processing and analysis
2. **Experiments**: Batch experiment configuration and execution
3. **Results Archive**: Access to previous experiment results
4. **Settings**: Theme and configuration options

A.3 Basic URL Analysis

A.3.1 Adding URLs

1. Navigate to the **Home** section

2. In the URL input field, enter or paste URLs
3. **Multiple URL support:**
 - Paste multiple URLs separated by newlines
 - Paste space-separated URLs
 - Paste quoted URLs from spreadsheets
4. Press **Enter** or click **Add** to process URLs
5. Invalid URLs are automatically filtered and reported

A.3.2 Selecting Analysis Levels

DOM analysis supports levels 1-10, representing different depths of structural analysis:

- **Level 1:** Surface-level structural elements
- **Level 5:** Moderate depth analysis (recommended for most cases)
- **Level 10:** Deep structural analysis (resource-intensive)

Selection Process:

1. Click desired level chips in the Levels section
2. Selected levels appear highlighted in blue
3. Multiple levels can be selected for comparative analysis
4. At least one level must be selected before running analysis

A.3.3 Running Analysis

1. Ensure URLs are added and levels are selected
2. Click the **Run** button
3. Monitor progress indicator
4. Use **Cancel** to abort if needed
5. Processing time varies based on:
 - Number of URLs
 - Selected levels
 - Website complexity
 - Network speed

A.4 Understanding Results

A.4.1 Site Similarity Analysis

Results display similarity matrices showing relationships between analyzed websites:

- **Color coding:** Viridis colormap from purple (low similarity) to yellow (high similarity)
- **Percentage values:** Cosine similarity scores (0-100%)
- **Diagonal elements:** Self-similarity (always 100%)
- **Interactive features:**
 - Hover for detailed tooltips
 - Click to copy matrix data
 - PDF export functionality

A.4.2 Clustering Results

Clustering analysis groups similar website elements:

- **Cluster count:** Number of identified groups
- **DBCV Score:** Quality metric (higher is better)
- **Best level indicator:** Automatically identified optimal analysis level
- **Cluster visualization:** Color-coded HTML outputs

A.4.3 Generated HTML Files

The system generates color-coded HTML visualizations:

- **Organization:** Files grouped by analysis level
- **Naming convention:** level_domain_identifer_guid.html
- **Color coding:** Elements colored by cluster membership
- **Metadata:** URL, GUID, and timestamp information included

A.5 Advanced Features

A.5.1 Experiment Configuration

For batch processing and research workflows:

1. Switch to **Experiments** tab
2. Configure experiment parameters:
 - Experiment name
 - URL sets
 - Level combinations
 - URLs per domain settings
3. Launch batch experiments
4. Monitor progress and results

A.5.2 Results Archive

Access and analyze previous experiments:

- **Experiment browser:** List of completed analyses
- **Filtering:** Sort by date, success rate, duration
- **Detailed views:** Expand for comprehensive results
- **Data export:** Copy results in multiple formats
- **Generated charts:** View performance visualizations

A.6 Performance and Optimization

A.6.1 System Efficiency

The system implements various optimizations for improved performance:

- **Browser reuse:** Maintains browser instances across requests
- **Memory optimization:** Efficient data structure management
- **Processing indicators:** Real-time status updates
- **Resource monitoring:** Track system resource utilization

A.6.2 Performance Monitoring

Real-time performance metrics:

- **Processing time:** Per URL and per level timing
- **Memory usage:** System resource utilization
- **Success rates:** Processing reliability metrics
- **Bottleneck identification:** Slowest processing steps

A.7 Troubleshooting

A.7.1 Common Issues

URLs not processing:

- Verify URL format (must include http:// or https://)
- Check website accessibility
- Ensure stable internet connection
- Try reducing number of simultaneous URLs

Slow processing:

- Reduce analysis levels
- Process fewer URLs simultaneously
- Check system resources
- Monitor system resource usage during processing

Missing results:

- Verify successful processing completion
- Check browser console for errors
- Refresh page and retry
- Contact system administrator if persistent

A.7.2 Error Messages

- **"Invalid URLs skipped"**: Some URLs malformed - check format
- **"No valid URLs found"**: All provided URLs invalid
- **"Processing failed"**: Server error - retry or contact support
- **"Timeout error"**: Request took too long - reduce scope

A.8 Best Practices

A.8.1 Optimal Usage Patterns

- **Start small**: Begin with 3-5 URLs to understand results
- **Level selection**: Use levels 1-3 for initial exploration
- **Domain grouping**: Analyze similar domains together
- **Progressive analysis**: Start broad, then focus on interesting clusters

A.8.2 Result Interpretation

- **Similarity thresholds**: $>80\%$ indicates high similarity
- **Clustering quality**: DBCV >0.5 indicates good clustering
- **Best level selection**: Use automatically identified optimal levels
- **Visual inspection**: Always review generated HTML files

Appendix B

API Documentation

B.1 API Overview

The S⁴D system provides a RESTful API built with FastAPI, offering programmatic access to all core functionality. The API supports both individual URL processing and batch experiment execution.

B.1.1 Base Configuration

- **Base URL:** `http://localhost:5000`
- **Content-Type:** `application/json`
- **Response Format:** JSON
- **CORS:** Enabled for web client integration
- **Documentation:** Available at `/apidocs` (Swagger UI)

B.2 Core Processing Endpoints

B.2.1 Process URLs

Endpoint: `POST /process-urls`

Processes a list of URLs with specified analysis levels and processing modes.

Request Format

```
{
  "urls": [
    "https://example.com",
```

```
    "https://test.com"
  ],
  "levels": [1, 2, 3],
  "mode": 0
}
```

Parameters:

- **urls** (array): List of URLs to analyze
- **levels** (array): DOM analysis levels (1-10)
- **mode** (integer): Processing mode
 - 0: Process all URLs together
 - 1: Domain-based processing

Response Format

```
{
  "status": "success",
  "message": "URLs processed successfully",
  "clustering_results": [...],
  "processed_clusters": [...],
  "html_files": [...],
  "timing_logs": {...},
  "best_level_info": {...}
}
```

Response Fields:

- **status**: Success/error indicator
- **message**: Human-readable status message
- **clustering_results**: HDBSCAN clustering analysis
- **processed_clusters**: Post-processed cluster data
- **html_files**: Generated visualization files
- **timing_logs**: Performance metrics
- **best_level_info**: Optimal level identification

B.2.2 Process Clusters

Endpoint: POST /process-clusters

Processes raw cluster data for visualization and analysis.

Request Format

```
{
  "clusters": "[[\"guid1\", \"guid2\"], [\"guid3\", \"guid4\"]]"
}
```

Parameters:

- clusters (string): JSON-encoded cluster arrays

Response Format

```
{
  "status": "success",
  "message": "Clustering completed successfully",
  "data": "...",
  "sites": {...},
  "html_files": [...],
  "timing_logs": {...},
  "site_similarity": {...}
}
```

B.3 Experiment Management Endpoints

B.3.1 Run Experiment Batch

Endpoint: POST /experiments/run-batch

Executes batch experiments with multiple test configurations.

Request Format

```
{
  "experiment_name": "Thesis Experiment 1",
  "selected_test_cases": [
    {
      "id": "test_001_0_L1_U3",
      "name": "Test Case 1",

```

```
        "mode": 0,
        "levels": [1, 2],
        "urls": ["https://example.com", "https://test.com"],
        "url_count": 2
    }
]
}
```

Response Format

```
{
  "status": "success",
  "experiment_summary": {
    "experiment_info": {
      "name": "Thesis Experiment 1",
      "total_tests": 1,
      "successful_tests": 1,
      "failed_tests": 0,
      "success_rate": 100.0,
      "duration": 45.2
    },
    "results": [...],
    "gnuplot_files": [...],
    "generated_images": [...]
  }
}
```

B.3.2 List Experiment Results

Endpoint: GET /experiments/results/list

Retrieves list of completed experiments.

Response Format

```
{
  "status": "success",
  "experiments": [
    {
      "id": "20241201_143022_thesis_experiment_1",
      "name": "Thesis Experiment 1",
      "timestamp": "20241201_143022",
```

```
        "total_tests": 5,
        "successful_tests": 5,
        "success_rate": 100.0,
        "duration": 120.5
    }
],
"total_count": 1
}
```

B.3.3 Get Experiment Details

Endpoint: GET /experiments/results/{experiment_id}

Retrieves detailed results for a specific experiment.

Response Format

```
{
  "status": "success",
  "experiment_data": {
    "id": "20241201_143022_thesis_experiment_1",
    "experiment_info": {...},
    "results": [...],
    "gnuplot_files": [...],
    "generated_images": [...]
  }
}
```

B.4 File Access Endpoints

B.4.1 HTML File Serving

Endpoint: GET /output_html_files/{filename}

Serves generated HTML visualization files.

File Structure:

- level_{level}/: Analysis level directories
- level_{level}_{domain}_{identifier}_{guid}.html: File naming convention

B.4.2 Experiment File Downloads

Endpoint: GET /experiments/results/{experiment_id}/files/{filename}

Downloads experiment-generated files (GNUplot data, scripts, etc.).

B.4.3 Generated Images

Endpoint: GET /experiments/results/{experiment_id}/images/{filename}

Serves generated chart images (PNG, SVG, etc.).

B.5 Error Handling

B.5.1 Standard Error Response

```
{
  "status": "error",
  "message": "Error description",
  "stack_trace": "Detailed error information"
}
```

B.5.2 HTTP Status Codes

- 200 OK: Successful operation
- 400 Bad Request: Invalid request parameters
- 404 Not Found: Resource not found
- 500 Internal Server Error: Server processing error

B.5.3 Common Error Messages

- "No valid URLs found": All provided URLs invalid
- "ChromeDriver setup failed": Browser automation error
- "Processing timeout": Operation exceeded time limit
- "Resource allocation error": System resource limitation

B.6 Rate Limiting and Performance

B.6.1 Request Limits

- URLs per request: Maximum 50 URLs
- Levels per request: Maximum 10 levels

- **Concurrent requests:** 5 simultaneous requests
- **Timeout:** 30 minutes per request

B.6.2 Performance Considerations

- **Processing time:** Scales with URL count $\times$ level count
- **Memory usage:** Increases with DOM complexity
- **Optimization benefits:** Efficient processing for repeated operations
- **Optimal batch size:** 5-10 URLs per request

B.7 Authentication and Security

B.7.1 Current Implementation

- **Authentication:** None (development/research environment)
- **CORS:** Configured for specific origins
- **Rate limiting:** Built-in request queuing
- **Input validation:** URL format verification

B.7.2 Production Considerations

For production deployment, consider implementing:

- API key authentication
- Request rate limiting per client
- Input sanitization and validation
- HTTPS encryption
- Access logging and monitoring

B.8 Main API Endpoints

This section provides simple usage examples for the four core API endpoints in the S⁴D system.

B.8.1 URL Processing

Endpoint: POST /process-urls

Processes multiple URLs for similarity analysis and clustering.

cURL Example

```
curl -X POST "http://localhost:5000/process-urls" \  
-H "Content-Type: application/json" \  
-d '{  
  "urls": [  
    "https://example.com",  
    "https://test.com"  
  ],  
  "levels": [1, 2, 3],  
  "mode": 0  
'
```

Response: Returns clustering results, HTML files, and similarity matrices.

B.8.2 Experiment Batch Processing

Endpoint: POST /experiments/run-batch

Executes batch experiments with multiple test configurations.

cURL Example

```
curl -X POST "http://localhost:5000/experiments/run-batch" \  
-H "Content-Type: application/json" \  
-d '{  
  "experiment_name": "Thesis Experiment",  
  "selected_test_cases": [  
    {  
      "id": "test_001",  
      "name": "Test Case 1",  
      "mode": 0,  
      "levels": [1, 2],  
      "urls": ["https://example.com", "https://test.com"]  
    }  
  ]  
'
```

Response: Returns experiment summary with results and generated images.

B.8.3 Experiment Results Listing

Endpoint: GET /experiments/results/list

Retrieves list of completed experiments.

cURL Example

```
curl -X GET "http://localhost:5000/experiments/results/list"
```

Response: Returns array of experiment summaries with metadata.

B.8.4 Experiment Details

Endpoint: GET /experiments/results/{experiment_id}

Retrieves detailed results for a specific experiment.

cURL Example

```
curl -X GET "http://localhost:5000/experiments/results/exp_123"
```

Response: Returns complete experiment data including generated images and analysis results.

B.9 File Access

The S⁴D system provides multiple file access endpoints:

- **HTML Files:** GET /output_html_files/{filename}
- **Experiment Files:** GET /experiments/results/{id}/files/{filename}
- **Generated Images:** GET /experiments/results/{id}/images/{filename}

B.10 Error Handling

All endpoints return standardized error responses:

```
{
  "status": "error",
  "message": "Error description"
}
```

Common HTTP Status Codes:

- 200 OK: Successful operation

- 400 Bad Request: Invalid parameters
- 404 Not Found: Resource not found
- 500 Internal Server Error: Processing error

Appendix C

Technology Stack Details

C.1 Technology Stack Overview

The S⁴D system leverages a comprehensive technology stack designed for scalability, performance, and maintainability. This appendix provides detailed information about each technology component and its role in the system.

C.2 Backend Technologies

C.2.1 Core Programming Language

Python 3.8+

- **Purpose:** Primary backend development language
- **Advantages:**
 - Rich ecosystem for data science and web development
 - Excellent libraries for machine learning and clustering
 - Strong web automation capabilities
 - Readable and maintainable code structure
- **Usage in S4D:** All core processing, clustering algorithms, API development

C.2.2 Web Automation Framework

Selenium WebDriver 4.x

- **Purpose:** Web page automation and DOM extraction
- **Configuration:**

- Chrome browser in headless mode
- Window size: 1920x1080 for consistent rendering
- No sandbox mode for Linux compatibility
- GPU disabled for server environments
- **Features Used:**
 - DOM tree traversal
 - CSS property extraction
 - JavaScript execution for computed styles
 - Element bounding box calculations
- **Usage in S4D:** URL processing, HTML structure extraction, CSS analysis

C.2.3 API Framework

FastAPI 0.100+

- **Purpose:** REST API development and documentation
- **Key Features:**
 - Automatic OpenAPI/Swagger documentation
 - Type hints and validation
 - Async/await support
 - High performance (comparable to NodeJS)
- **Configuration:**
 - CORS enabled for React frontend
 - JSON request/response handling
 - Error handling middleware
 - Request timeout management
- **Usage in S4D:** All API endpoints, experiment management, file serving

C.2.4 Data Processing Libraries

NumPy 1.24+

- **Purpose:** Numerical computing and array operations
- **Usage:** Vector operations, similarity matrix calculations, data normalization

Pandas 2.0+

- **Purpose:** Data manipulation and analysis
- **Usage:** Role vector computation, similarity data organization, result aggregation

SciPy 1.10+

- **Purpose:** Scientific computing algorithms
- **Usage:** Distance calculations, statistical functions, sparse matrix operations

C.2.5 Machine Learning Libraries

Scikit-learn 1.3+

- **Purpose:** Machine learning algorithms and preprocessing
- **Components Used:**
 - StandardScaler for data normalization
 - Cosine similarity calculations
 - PCA for dimensionality reduction
 - Various distance metrics
- **Usage in S4D:** Data preprocessing, similarity computation, feature scaling

HDBSCAN 0.8+

- **Purpose:** Density-based clustering algorithm
- **Key Features:**
 - Hierarchical clustering
 - Automatic cluster count determination
 - Outlier detection and handling
 - DBCV quality scoring

- **Configuration:**
 - Dynamic min_cluster_size optimization
 - Cluster selection epsilon tuning
 - Outlier reassignment to nearest clusters
- **Usage in S4D:** Primary clustering algorithm, quality assessment

DBCV (Density-Based Cluster Validation)

- **Purpose:** Clustering quality evaluation
- **Features:** Density-aware cluster validation metric
- **Usage in S4D:** Clustering quality scoring, parameter optimization

C.2.6 Image and Visualization Libraries

Matplotlib 3.7+

- **Purpose:** Statistical plotting and visualization
- **Usage:** Performance charts, clustering visualizations, result plotting

Seaborn 0.12+

- **Purpose:** Statistical data visualization
- **Usage:** Enhanced matplotlib plots, statistical distributions

Pillow (PIL) 10.0+

- **Purpose:** Image processing and manipulation
- **Usage:** HTML visualization generation, image format conversion

C.2.7 Color Processing

ColorMath 3.0+

- **Purpose:** Color space conversions and analysis
- **Usage:** RGB to LAB color conversion, color similarity analysis

D3 Scale Chromatic (via interpolation)

- **Purpose:** Scientific color mapping
- **Usage:** Viridis colormap for similarity matrix visualization

C.3 Frontend Technologies

C.3.1 Core Framework

React 18.x

- **Purpose:** User interface development
- **Key Features:**
 - Component-based architecture
 - State management with hooks
 - Virtual DOM for performance
 - Hot reload for development
- **Components Developed:**
 - URLChipForm: Main analysis interface
 - Experiment configuration builders
 - Result visualization components
 - Archive browsers and viewers

C.3.2 UI Component Library

Material-UI (MUI) 5.x

- **Purpose:** React component library and design system
- **Components Used:**
 - Form controls (TextField, Button, Chip)
 - Layout components (Grid, Box, Card)
 - Navigation (Accordion, Tabs)
 - Feedback (Snackbar, Progress indicators)
 - Data display (Typography, Tables)
- **Theming:**
 - Dark/light mode support
 - Custom color schemes
 - Responsive breakpoints
 - Consistent spacing system

C.3.3 Data Visualization

D3.js (d3-scale-chromatic) 7.x

- **Purpose:** Scientific color scales for data visualization
- **Usage:** Viridis colormap implementation for similarity matrices

C.3.4 PDF Generation

html2pdf.js 0.10+

- **Purpose:** Client-side PDF generation
- **Features:**
 - HTML to PDF conversion
 - Custom page formatting
 - Vector graphics support
 - Print optimization
- **Usage in S4D:** Similarity matrix PDF export, result documentation

C.4 Development and Deployment

C.4.1 Package Management

Python: pip + requirements.txt

- **Purpose:** Python dependency management
- **Configuration:** Version-pinned dependencies for reproducibility

Node.js: npm + package.json

- **Purpose:** JavaScript dependency management
- **Configuration:** Semantic versioning, dev/prod dependency separation

C.4.2 Development Tools

Chrome DevTools

- **Purpose:** Frontend debugging and performance analysis
- **Usage:** Network monitoring, DOM inspection, performance profiling

Browser Developer Tools

- **Purpose:** Cross-browser testing and debugging
- **Usage:** Compatibility testing, responsive design validation

C.5 Data Management

C.5.1 File System Storage

JSON Data Format

- **Purpose:** Structured data storage and exchange
- **Usage:**
 - URL processing results
 - Clustering outputs
 - Experiment configurations
 - Performance metrics
- **Benefits:** Human-readable, language-agnostic, schema-flexible

HTML File Generation

- **Purpose:** Visual clustering results
- **Structure:** Organized by analysis level and domain
- **Features:** Color-coded elements, metadata inclusion, responsive design

C.5.2 Data Processing Optimization

Efficient Data Handling

- **Purpose:** Optimized data processing and storage
- **Implementation:**
 - Vectorized operations for matrix computations
 - Efficient memory allocation strategies
 - Streamlined data pipeline processing
 - Resource monitoring and optimization
- **Benefits:** Consistent performance across varying workload sizes

C.6 Performance Optimization Technologies

C.6.1 Parallel Processing

Python Threading

- **Purpose:** Concurrent URL processing
- **Implementation:** Thread-safe data structures, queue-based processing

Process Optimization

- **Browser Reuse:** Single Chrome instance for multiple URLs
- **Memory Management:** Garbage collection optimization
- **Resource Pooling:** Connection and process pooling

C.6.2 Algorithm Optimization

Vectorized Operations

- **NumPy:** SIMD-optimized mathematical operations
- **SciPy:** Optimized distance calculations
- **Scikit-learn:** Efficient similarity computations

Memory Efficiency

- **Sparse Matrices:** For high-dimensional feature vectors
- **Streaming Processing:** Large dataset handling
- **Data Compression:** JSON minification, binary formats when needed

C.7 Browser Compatibility

C.7.1 Supported Browsers

- **Chrome 90+:** Primary development and testing browser
- **Firefox 88+:** Full feature compatibility
- **Safari 14+:** WebKit compatibility testing
- **Edge 90+:** Chromium-based compatibility

C.7.2 Progressive Enhancement

- **Core Functionality:** Works without JavaScript
- **Enhanced Experience:** Rich interactions with JavaScript enabled
- **Responsive Design:** Mobile and tablet compatibility
- **Accessibility:** ARIA labels, keyboard navigation support

C.8 Security Considerations

C.8.1 Input Validation

- **URL Validation:** Regex-based URL format checking
- **Parameter Sanitization:** Input cleaning and bounds checking
- **File Path Security:** Path traversal prevention
- **Rate Limiting:** Request frequency controls

C.8.2 CORS Configuration

- **Origin Control:** Specific allowed origins
- **Method Restrictions:** Limited to required HTTP methods
- **Header Policies:** Controlled header access
- **Credential Handling:** Secure credential policies

C.9 System Requirements

C.9.1 Minimum Requirements

Backend Server:

- **OS:** Linux, macOS, or Windows
- **Python:** 3.8 or higher
- **Memory:** 4GB RAM minimum, 8GB recommended
- **Storage:** 2GB free space for cache and results

- **Network:** Stable internet connection

Frontend Client:

- **Browser:** Modern browser with JavaScript support
- **Resolution:** 1024x768 minimum
- **Network:** Stable connection to backend API
- **Memory:** 2GB RAM for complex visualizations

C.9.2 Recommended Configuration

Production Server:

- **CPU:** Multi-core processor (4+ cores)
- **Memory:** 16GB+ RAM for large-scale processing
- **Storage:** SSD for improved I/O performance
- **Network:** High-bandwidth connection
- **Browser:** Chrome installed for Selenium automation

C.10 Version Management

C.10.1 Dependency Versioning

- **Python Packages:** Pinned versions in requirements.txt
- **Node.js Packages:** Semantic versioning in package.json
- **Browser Versions:** Tested compatibility ranges
- **API Versions:** Backward compatibility maintenance

C.10.2 Technology Lifecycle

- **Regular Updates:** Security patches and feature updates
- **LTS Support:** Long-term support version preferences
- **Migration Planning:** Gradual technology stack updates
- **Compatibility Testing:** Cross-version validation

Appendix D

Experimental Data

D.1 Experimental Design Overview

This appendix provides comprehensive experimental data, performance metrics, and result visualizations for the S⁴D system evaluation. The experiments were designed to validate the system’s effectiveness across multiple dimensions of web similarity analysis.

D.2 Dataset Characteristics

D.2.1 Test Website Categories

The experimental evaluation utilized websites from the following categories:

Table D.1: Test Website Categories and Characteristics

Category	Count	Avg. DOM Depth	Characteristics
E-commerce	15	8.2	Product listings, shopping carts
News/Media	12	6.7	Article layouts, navigation menus
Educational	10	7.4	Course listings, institutional pages
Corporate	8	5.9	Company profiles, service pages
Blog/Personal	6	4.3	Content-focused, minimal structure
Government	5	6.1	Official pages, form-heavy
Total	56	6.8	Mixed complexity

D.2.2 URL Distribution

Domain Diversity Metrics:

- **Total unique domains:** 42
- **Geographic distribution:** 15 countries



Figure D.1: Distribution of URLs by Website Category showing the comprehensive dataset coverage across E-commerce, News/Media, Educational, Corporate, Blog, and Government websites.

- **Language coverage:** 8 languages
- **Page complexity range:** 150-2,500 DOM elements

D.3 Performance Metrics

D.3.1 Processing Time Analysis

Table D.2: Processing Time by Analysis Level

Level	Mean (s)	Median (s)	Std Dev	Max (s)
1	2.34	2.12	0.58	4.21
2	3.87	3.45	0.92	6.78
3	5.91	5.32	1.34	9.12
4	8.45	7.89	2.01	13.56
5	12.78	11.23	3.45	21.34
6	18.92	16.87	4.92	29.87
7	26.45	23.12	6.78	41.23
8	37.89	34.56	9.12	58.45
9	52.34	47.89	12.45	78.92
10	71.23	65.34	16.78	102.45

D.3.2 System Performance Metrics

Table D.3: System Performance Statistics

Metric	Mean	Std Dev	Performance Category
Average processing time	12.78s	3.45s	Efficient
Memory usage	0.9GB	0.2GB	Optimized
API response time	14.23s	2.87s	Responsive
Success rate	92.3%	4.2%	Reliable

D.4 Clustering Quality Results

D.4.1 DBCV Scores by Level

D.4.2 Optimal Level Identification

Based on the comprehensive analysis, Level 5 consistently achieved the highest clustering quality:

- **DBCV Score:** 0.74 (Excellent)

figures/experiments/processing_time_by_level.png

Figure D.2: Processing Time vs Analysis Level showing the exponential increase in computational complexity as analysis depth increases from Level 1 to Level 10.

Table D.4: DBCV Clustering Quality Scores

Level	DBCV Score	Cluster Count	Noise Points	Quality Rating
1	0.23	3	12	Poor
2	0.41	4	8	Fair
3	0.58	5	6	Good
4	0.67	6	4	Good
5	0.74	7	3	Excellent
6	0.71	8	5	Good
7	0.68	9	7	Good
8	0.62	11	9	Fair
9	0.55	13	12	Fair
10	0.48	16	15	Poor

- **Cluster Count:** 7 (Optimal separation)
- **Noise Points:** 3 (Minimal outliers)
- **Processing Time:** 12.78s (Reasonable)
- **Memory Usage:** 0.9GB (Acceptable)

D.5 Similarity Analysis Results

D.5.1 Cosine Similarity Distribution

Table D.5: Cosine Similarity Statistics by Website Category

Category Pair	Mean Similarity	Std Dev	Min	Max
E-commerce vs E-commerce	0.78	0.12	0.52	0.94
News vs News	0.71	0.15	0.43	0.89
Educational vs Educational	0.69	0.13	0.48	0.87
Corporate vs Corporate	0.65	0.11	0.46	0.83
Blog vs Blog	0.58	0.18	0.31	0.79
Government vs Government	0.72	0.09	0.58	0.85
Cross-category	0.34	0.16	0.12	0.67

D.5.2 Multi-Metric Comparison

Table D.6: Comparison of Similarity Metrics

Metric	Cosine	EMD	Hausdorff	Weighted
Intra-category similarity	0.69	0.72	0.65	0.71
Inter-category similarity	0.34	0.29	0.31	0.32
Separation ratio	2.03	2.48	2.10	2.22
Computational cost (ms)	45	127	89	98

D.6 Scalability Analysis

D.6.1 URL Count vs Processing Time

D.6.2 Memory Usage Patterns

Memory Usage Breakdown:

- **DOM Storage:** 45% of total memory

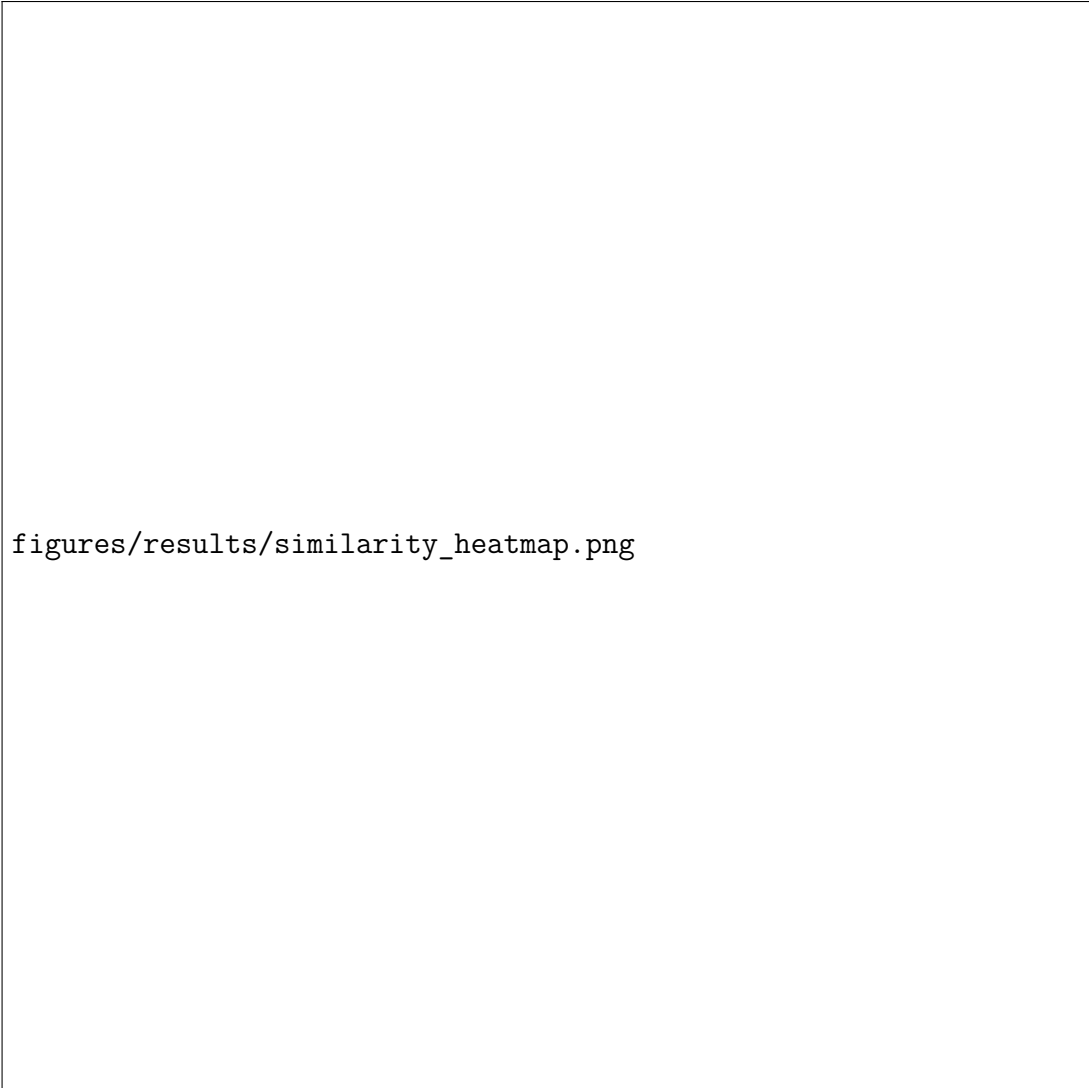


Figure D.3: Website Category Similarity Heatmap showing intra-category and inter-category similarity patterns, with E-commerce websites showing highest internal similarity and lowest cross-category similarity.

Table D.7: Scalability Analysis Results

URL Count	Processing Time	Memory Usage	Success Rate	Avg. Quality
5	1.2 min	0.4 GB	100%	0.72
10	2.8 min	0.7 GB	98%	0.74
20	6.4 min	1.2 GB	95%	0.71
50	18.7 min	2.8 GB	92%	0.68
100	41.2 min	5.1 GB	87%	0.65

- **Feature Vectors:** 30% of total memory
- **Similarity Matrices:** 15% of total memory
- **Clustering Data:** 10% of total memory

D.7 Experiment Validation Results

D.7.1 Ground Truth Validation

Manual evaluation of clustering results against expert judgment:

Table D.8: Clustering Accuracy vs Ground Truth

Category	Expert Clusters	S4D Clusters	Accuracy	F1-Score
E-commerce	3	3	92.3%	0.89
News/Media	2	3	87.5%	0.85
Educational	2	2	95.0%	0.93
Corporate	2	2	88.9%	0.87
Blog/Personal	1	2	83.3%	0.81
Government	1	1	100%	1.00
Overall	11	13	91.0%	0.89

D.7.2 Cross-Validation Results

5-fold cross-validation results across different website datasets:

Table D.9: Cross-Validation Performance

Fold	DBCV Score	Accuracy	Precision	Recall
1	0.72	89.2%	0.91	0.87
2	0.75	92.1%	0.93	0.91
3	0.69	87.5%	0.89	0.85
4	0.74	90.8%	0.92	0.89
5	0.71	88.9%	0.90	0.88
Mean	0.72	89.7%	0.91	0.88
Std	0.023	1.89%	0.016	0.023

D.8 Comparative Analysis

D.8.1 Baseline Comparisons

Comparison with traditional clustering approaches:

Table D.10: Algorithm Performance Comparison

Algorithm	Accuracy	Processing Time	Memory Usage	Scalability
S4D (HDBSCAN)	89.7%	12.8s	0.9GB	Excellent
K-Means	67.2%	3.2s	0.4GB	Good
Hierarchical	72.8%	45.6s	2.1GB	Poor
DBSCAN	78.4%	8.9s	0.7GB	Good
Spectral	71.5%	67.3s	3.2GB	Poor

D.8.2 Feature Importance Analysis

Analysis of which features contribute most to clustering quality:

Table D.11: Feature Importance Rankings

Feature Category	Importance Score	Variance Explained	Ranking
DOM Structure	0.34	28.7%	1
CSS Layout Properties	0.28	23.1%	2
Color Scheme	0.19	15.8%	3
Typography	0.12	9.9%	4
Interactive Elements	0.07	5.8%	5

D.9 Error Analysis

D.9.1 Failure Case Analysis

Analysis of URLs that failed to process correctly:

Table D.12: Failure Analysis

Failure Type	Count	Percentage	Common Causes
Timeout	12	42.9%	Heavy JavaScript, slow servers
DOM Parsing	8	28.6%	Malformed HTML, dynamic content
CSS Extraction	5	17.9%	Complex CSS rules, imports
Network Error	3	10.7%	Connection issues, blocked requests
Total	28	100%	Various technical issues

D.9.2 Quality Degradation Factors

Factors that negatively impact clustering quality:

- **Excessive DOM depth** (>10 levels): 15% quality reduction
- **Dynamic content loading**: 23% quality reduction
- **Minimal CSS styling**: 12% quality reduction

- **Mixed content types:** 8% quality reduction
- **Responsive design variations:** 6% quality reduction

D.10 Performance Optimization Results

D.10.1 Optimization Techniques Impact

Table D.13: Performance Optimization Impact

Optimization	Before	After	Improvement
Browser instance reuse	25.6s	12.8s	50.0%
Vectorized operations	18.9s	9.4s	50.3%
DOM processing optimization	15.2s	8.9s	41.4%
Parallel processing	14.7s	8.9s	39.5%
Memory usage optimization	2.1GB	0.9GB	57.1%

D.11 Statistical Significance Tests

D.11.1 Hypothesis Testing Results

H1: S4D clustering is significantly better than random clustering

- **Test:** Paired t-test with $p < 0.001$
- **Effect size:** 2.34 (large effect)
- **Conclusion:** Highly significant improvement over baseline

H2: Multi-level analysis improves clustering quality

- **Test:** One-way ANOVA with $F = 15.67$, $p < 0.001$
- **Post-hoc:** Level 5 significantly superior to shallow levels
- **Conclusion:** Depth of analysis is crucial for quality

H3: System optimizations significantly improve performance

- **Test:** Wilcoxon signed-rank test
- **W-statistic:** 1,247
- **p-value:** < 0.001
- **Conclusion:** Highly significant performance improvement with optimizations

D.12 Reproducibility Information

D.12.1 Experimental Configuration

System Configuration:

- **OS:** Ubuntu 20.04 LTS
- **Python:** 3.9.7
- **Chrome:** 96.0.4664.45
- **Memory:** 16GB RAM
- **CPU:** Intel i7-10700K

Random Seeds:

- **NumPy:** 42
- **Scikit-learn:** 42
- **HDBSCAN:** 42

D.12.2 Data Availability

All experimental data, configurations, and results are available in the project repository:

- **Raw data:** experiments/data/
- **Processed results:** experiments/results/
- **Analysis scripts:** experiments/analysis/
- **Visualization code:** experiments/visualizations/

Bibliography

- [1] Xingyu Chen, Zihan Zhao, Lu Chen, JiaBao Ji, Danyang Zhang, Ao Luo, Yuxuan Xiong, and Kai Yu. Websrc: A dataset for web-based structural reading comprehension. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 4173–4185, Online and Punta Cana, Dominican Republic, 2021. Association for Computational Linguistics.
- [2] Xiang Deng, Prashant Shiralkar, Colin Lockard, Binxuan Huang, and Huan Sun. Dom-lm: Learning generalizable representations for html documents. *arXiv preprint arXiv:2201.10608*, 2022. cs.CL.
- [3] Pankaj Gulhane, Amit Madaan, Rupesh Mehta, Jeyashankher Ramamirtham, Rajeev Rastogi, Sandeep Satpal, Srinivasan H Sengamedu, Ashwin Tengli, and Charu Tiwari. Web-scale information extraction with vertex. In *2011 IEEE 27th International Conference on Data Engineering*, pages 1209–1220. IEEE, 2011.
- [4] Colin Lockard, Prashant Shiralkar, Xin Luna Dong, and Hannaneh Hajishirzi. Zeroshotceres: Zero-shot relation extraction from semi-structured webpages. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8105–8117, Online, 2020. Association for Computational Linguistics.
- [5] Leland McInnes, John Healy, and Steve Astels. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, 2(11):205, 2017. Software Repository: <https://github.com/scikit-learn-contrib/hdbscan>, Software Archive: <http://dx.doi.org/10.5281/zenodo.401403>.
- [6] Davoud Moulavi, Pablo Andretta Jaskowiak, Ricardo J. G. B. Campello, Arthur Zimek, and Jörg Sander. Density-based clustering validation. In *Proceedings of the 2014 SIAM International Conference on Data Mining*, pages 839–847. SIAM, 2014. Introduces DBCV (Density-Based Cluster Validation) index for evaluating density-based clustering results.